



Sichuan University -
Pittsburgh Institute

ECEo202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute



Interrupt vs Polling



Copyright © Ron Leishman * <http://ToonClips.com/9845>

Polling:

You pick up the phone every few seconds to check whether you are getting a call.

Interrupt vs Polling

Interrupt:

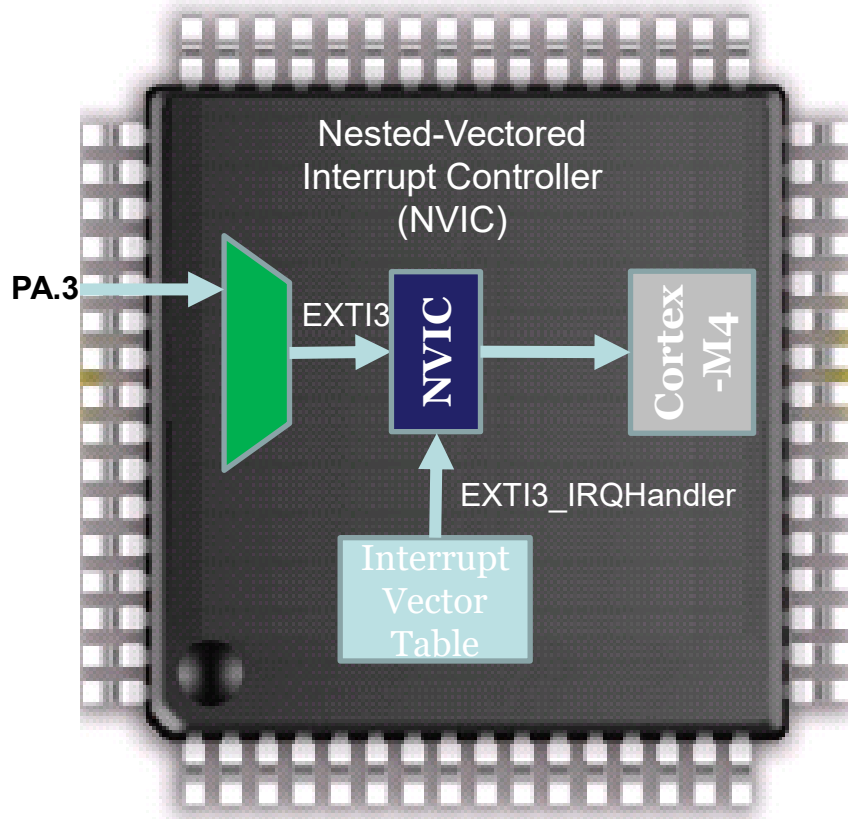
Do whatever you should do and pick up the phone **when it rings**.

- Hardware-triggered software action
- Respond to external events efficiently, while avoiding busy wait.
- When no events, the processor can run the main program or enter a sleep state to conserve energy



<http://www.istockphoto.com/>

Interrupt Vector Table

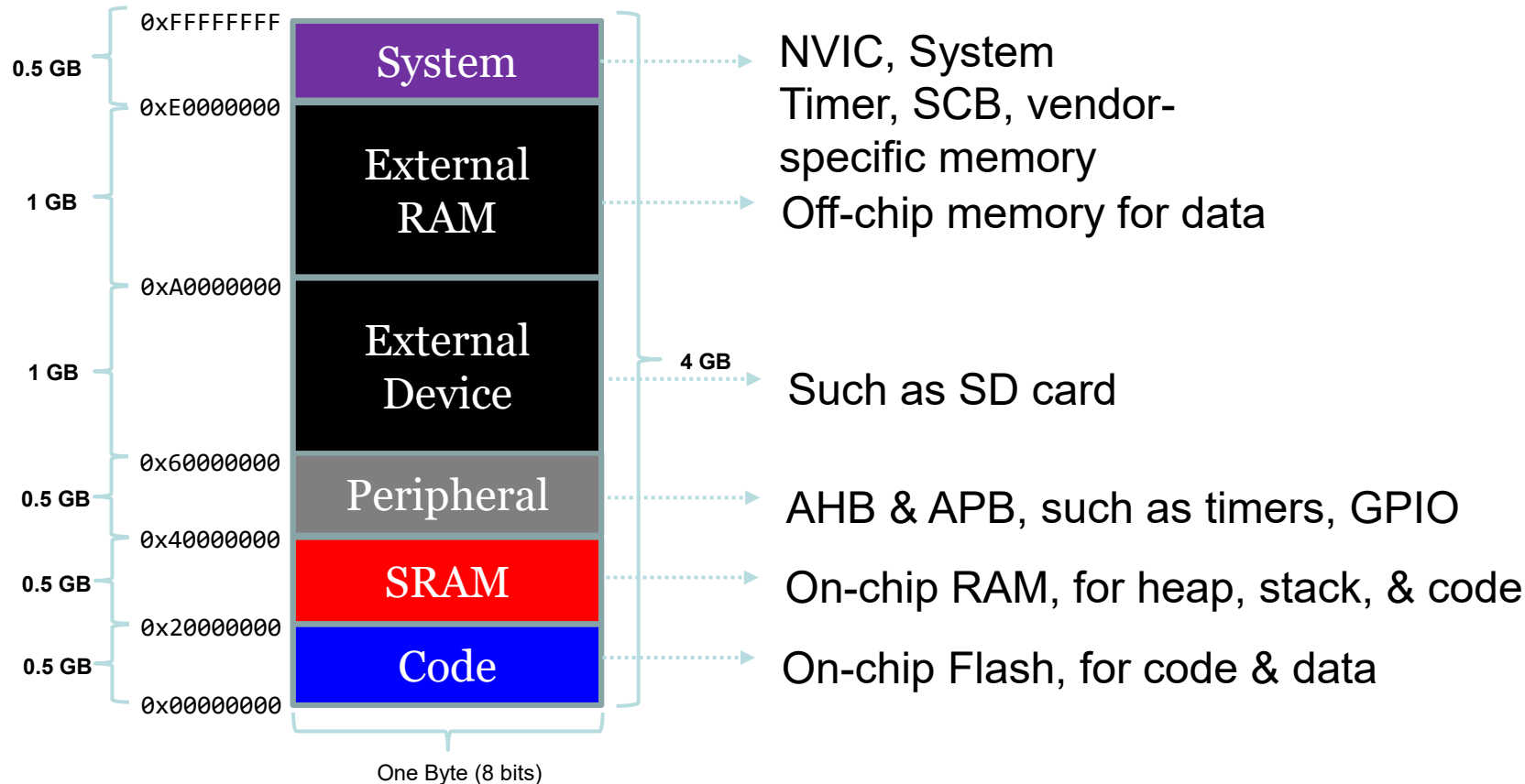


Interrupt Vector Table Address of ISR 1

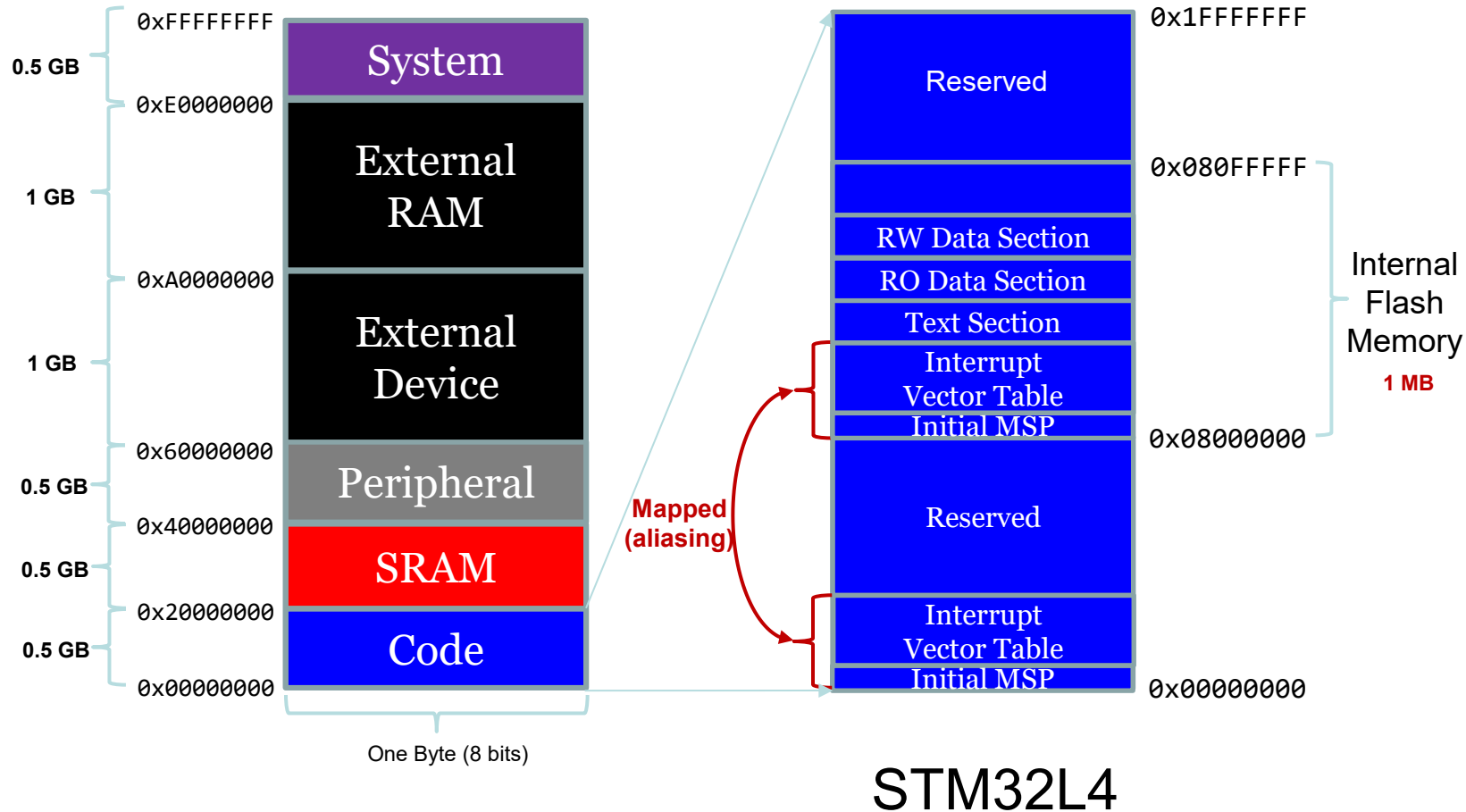
Interrupt Number (8 bits)	Memory Address of ISR (32 bits)
1	Interrupt Service Routine for interrupt 1
2	Interrupt Service Routine for interrupt 2
3	Interrupt Service Routine for interrupt 3
4	Interrupt Service Routine for interrupt 4
5	Interrupt Service Routine for interrupt 5
...	...

When interrupt x is triggered, jump to the ISR for interrupt x . ($1 \leq x \leq 255$)

Memory Map of Cortex-M4



Instruction Memory

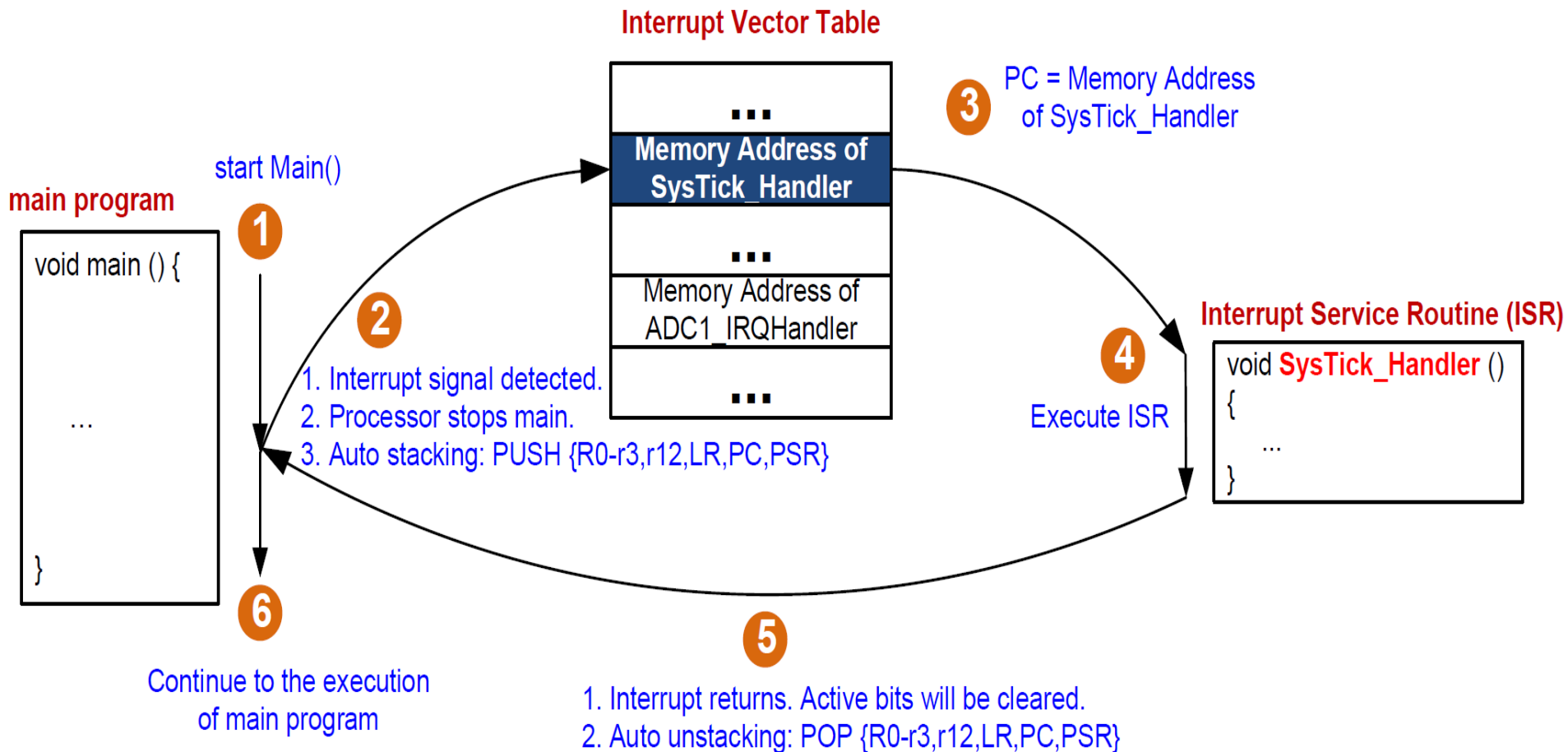


ISR Vector Table

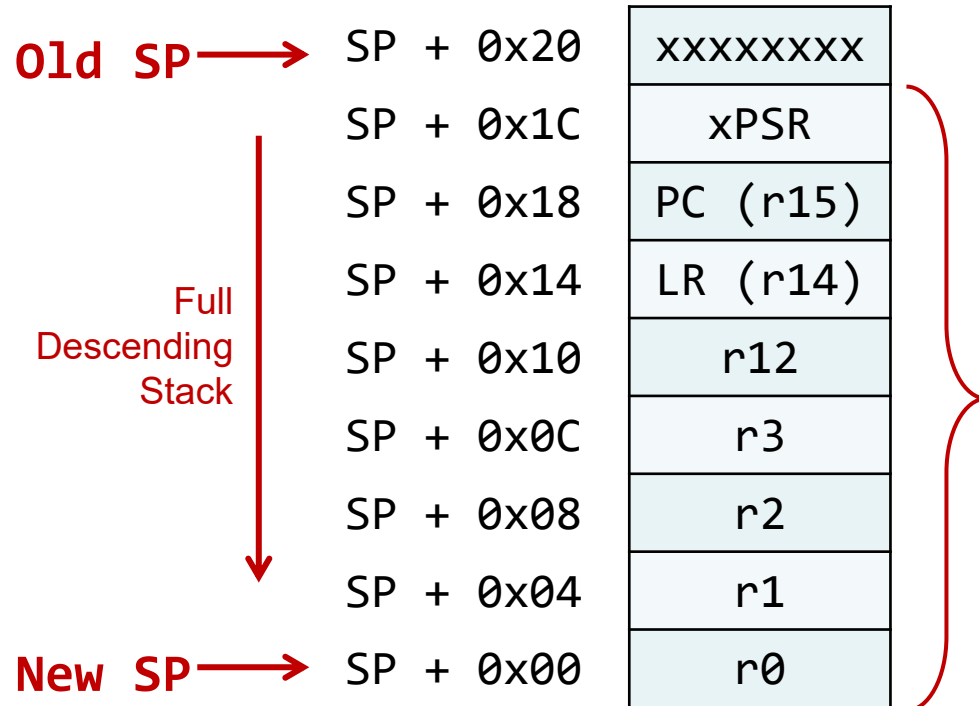
	0x00000074	DMA1_Channel4_IRQHandler	
	0x00000070	DMA1_Channel3_IRQHandler	
	0x0000006C	DMA1_Channel2_IRQHandler	
	0x00000068	DMA1_Channel1_IRQHandler	void DMA1_Channel1_IRQHandler () { ... }
	0x00000064	EXTI4_IRQHandler	
	0x00000060	EXTI3_IRQHandler	void EXTI1_Handler () { ... }
	0x0000005C	EXTI2_IRQHandler	
	0x00000058	EXTI1_IRQHandler	
	0x00000054	EXTI0_IRQHandler	void EXTI0_Handler () { ... }
	0x00000050	RCC_IRQHandler	
	0x0000004C	FLASH_IRQHandler	
	0x00000048	RTC_WKUP_IRQHandler	
	0x00000044	TAMPER_STAMP_IRQHandler	
	0x00000040	PVD_IRQHandler	
	0x0000003C	WWDG_IRQHandler	
	0x00000038	SysTick_Handler	void SysTick_Handler () { ... }
	0x00000034	PendSV_Handler	
	0x00000030	DebugMon_Handler	
	0x0000002C	SVC_Handler	void SVC_Handler () { ... }
	0x00000028	Reserved	
	0x00000024	Reserved	
	0x00000020	Reserved	
	0x0000001C	Reserved	
	0x00000018	UsageFault_Handler	
	0x00000014	BusFault_Handler	
	0x00000010	MemMange_Handler	
	0x0000000C	HardFault_Handler	void Reset_Handler () { ... main(); ... }
	0x00000008	NMI_Handler	
	0x00000004	Reset_Handler	Value to initialize the Program Counter (PC)
	0x00000000	Top_of_Stack	Value to initialize the Stack Pointer (SP)

System
Exceptions

Interrupt

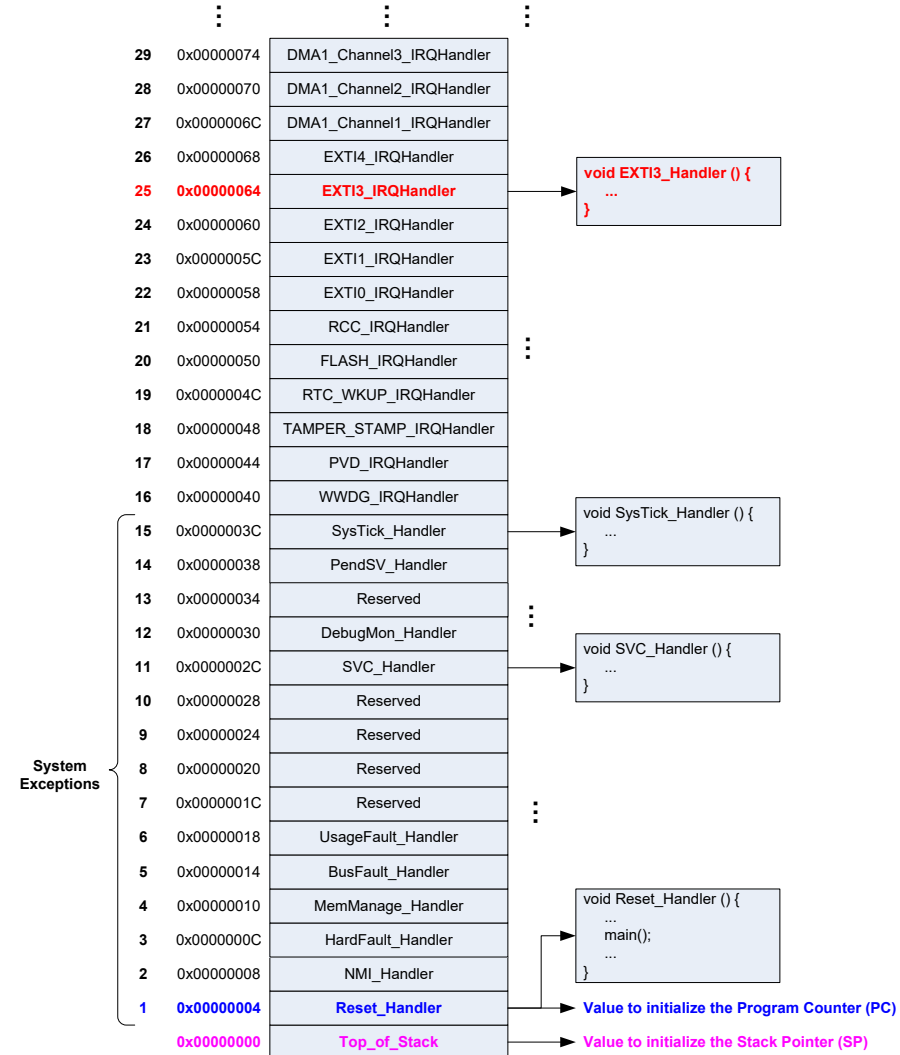
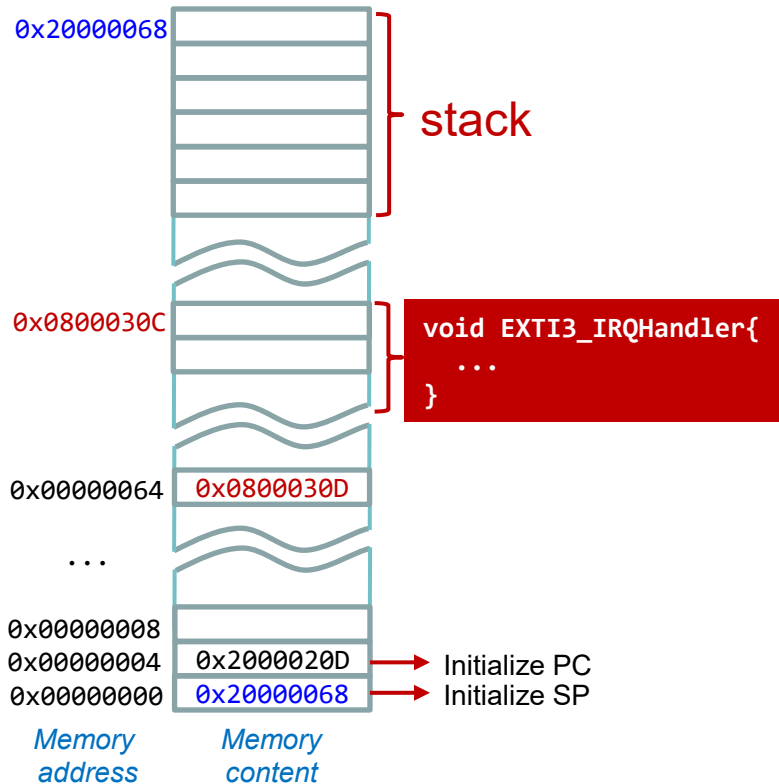


Stacking & Unstacking

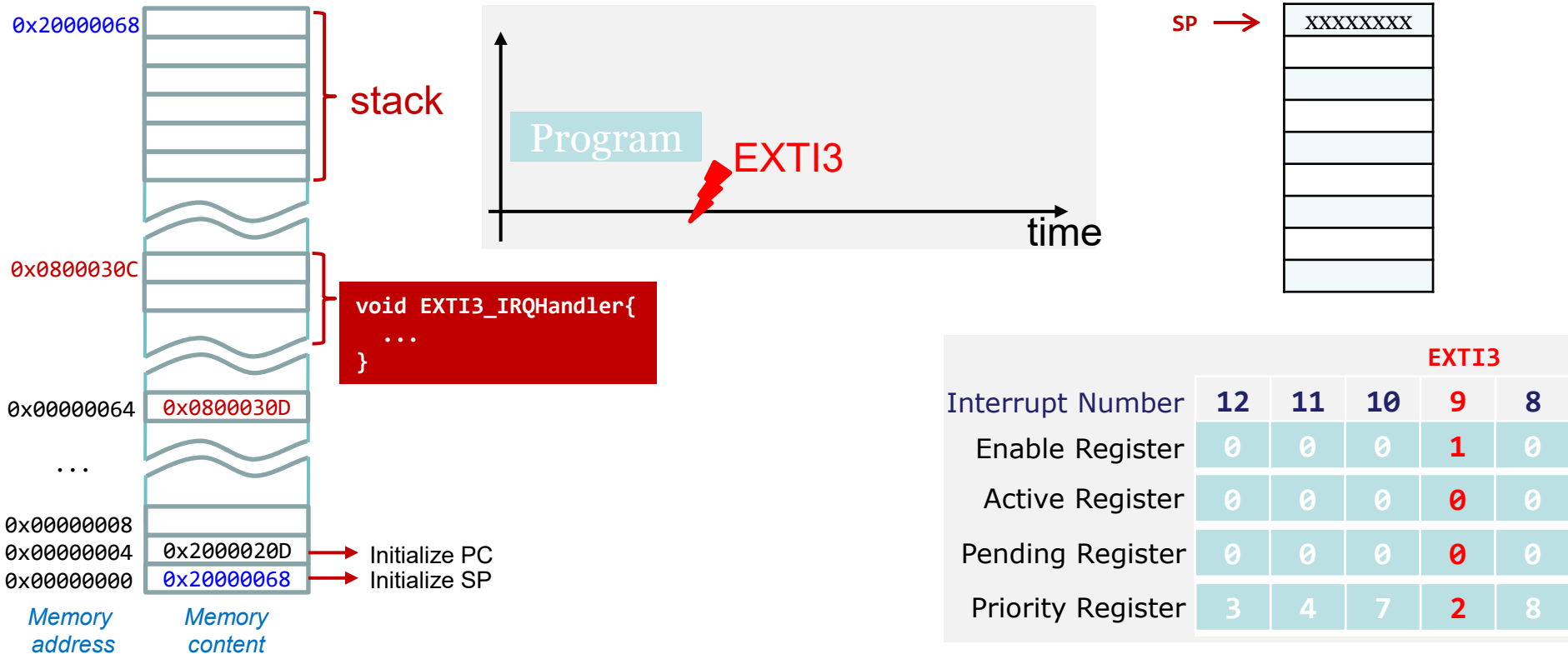


- **Stacking:** The processor automatically pushes these **eight** registers into the main stack before an interrupt handler **starts**
- **Unstacking:** The processor automatically pops these **eight** registers out of the main stack when an interrupt handler **exits**.

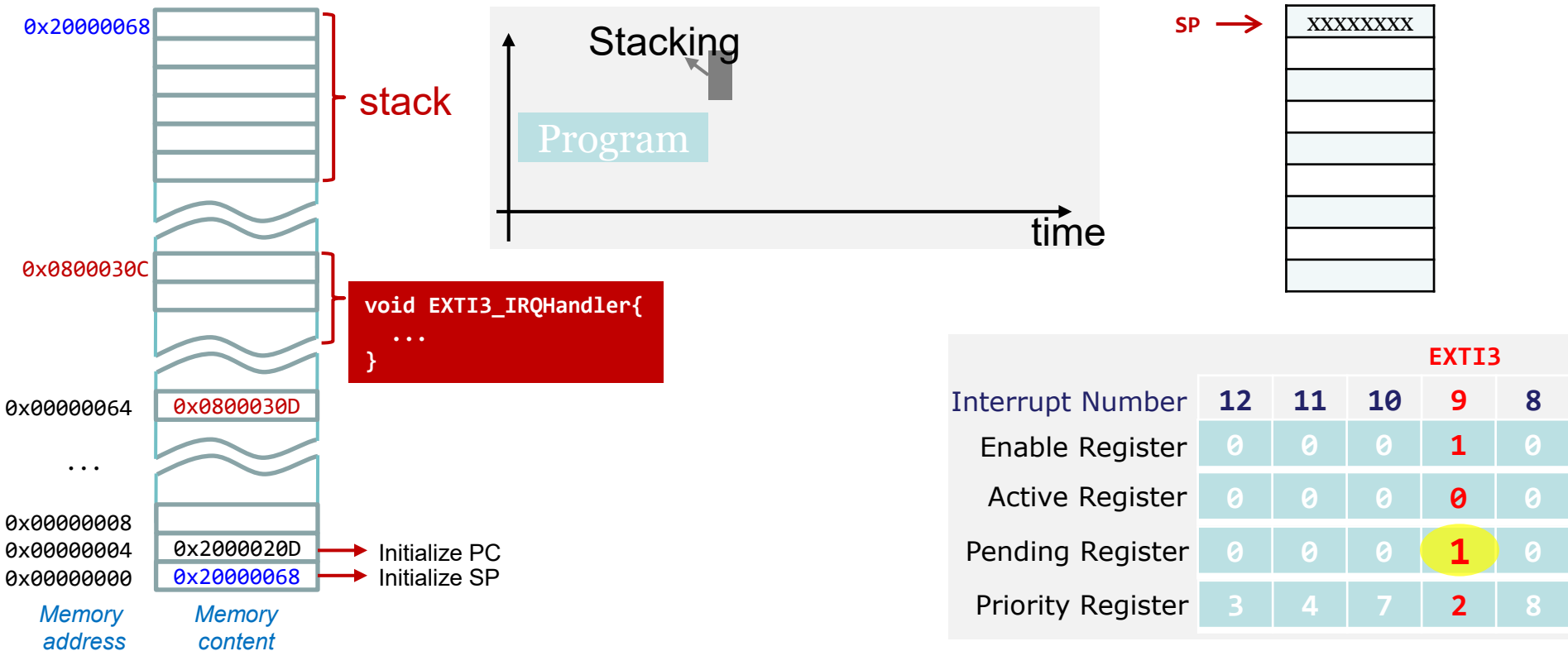
Interrupt Service Routine (ISR)



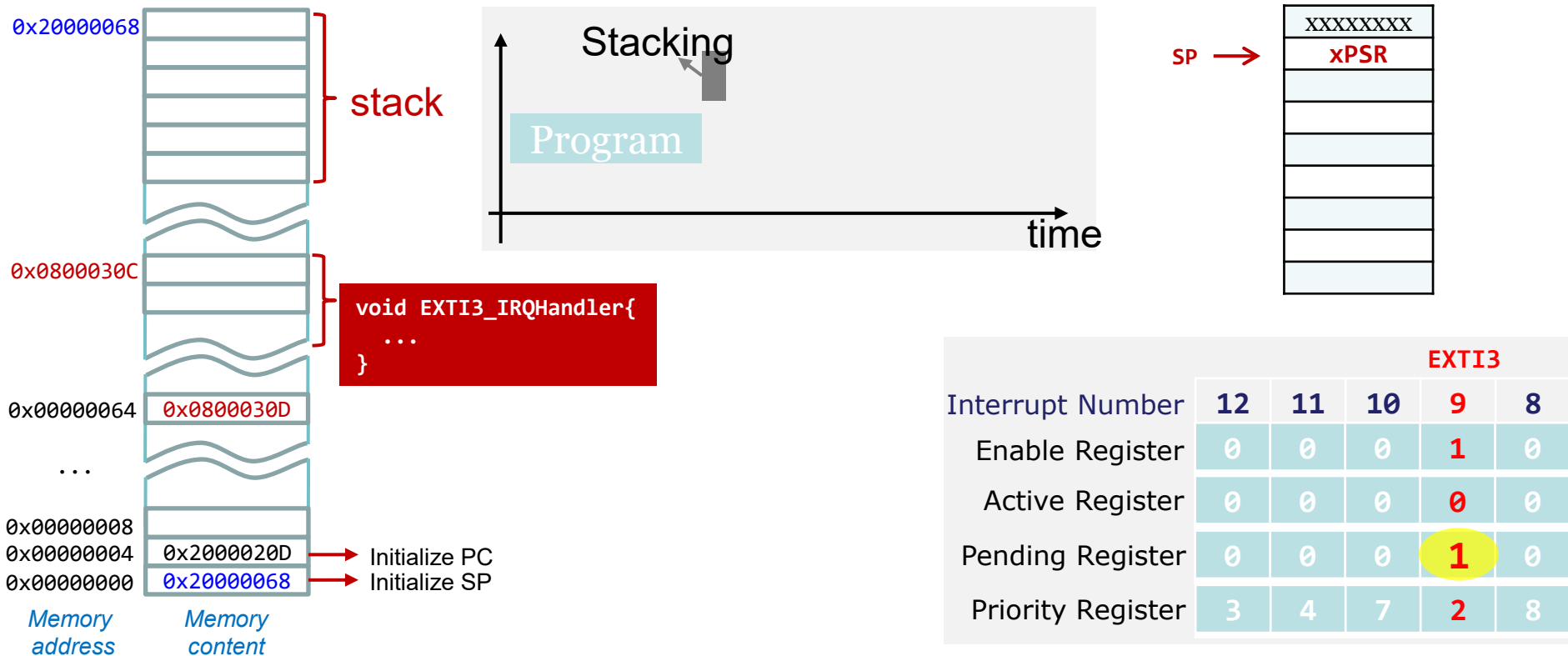
Single Interrupt



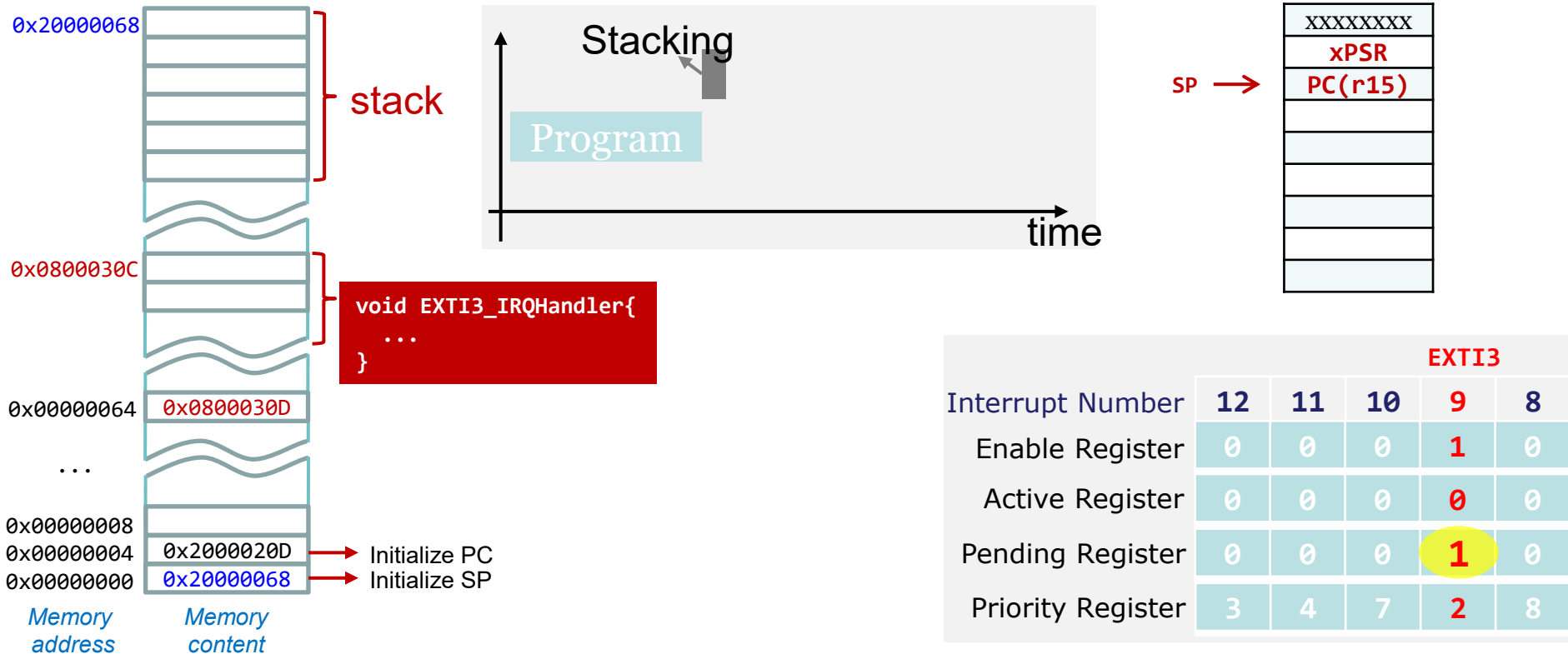
Single Interrupt



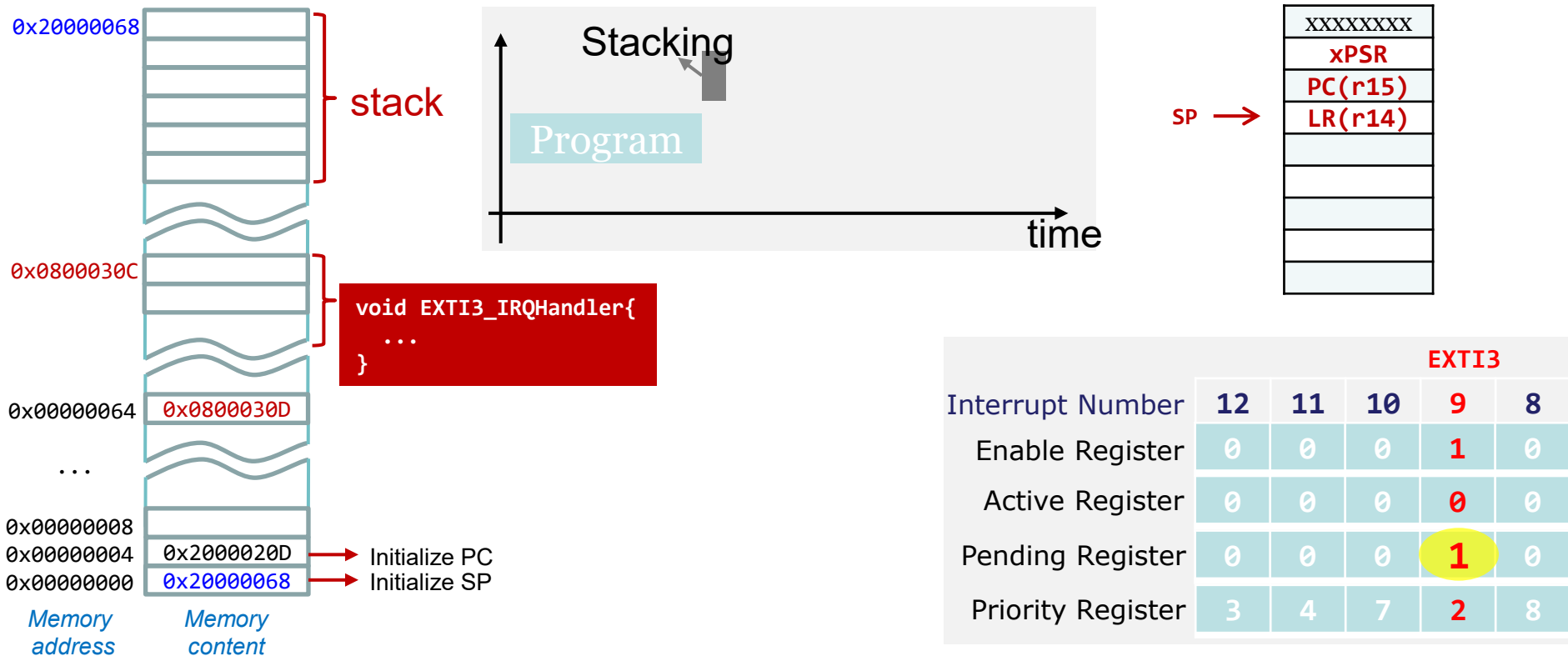
Single Interrupt



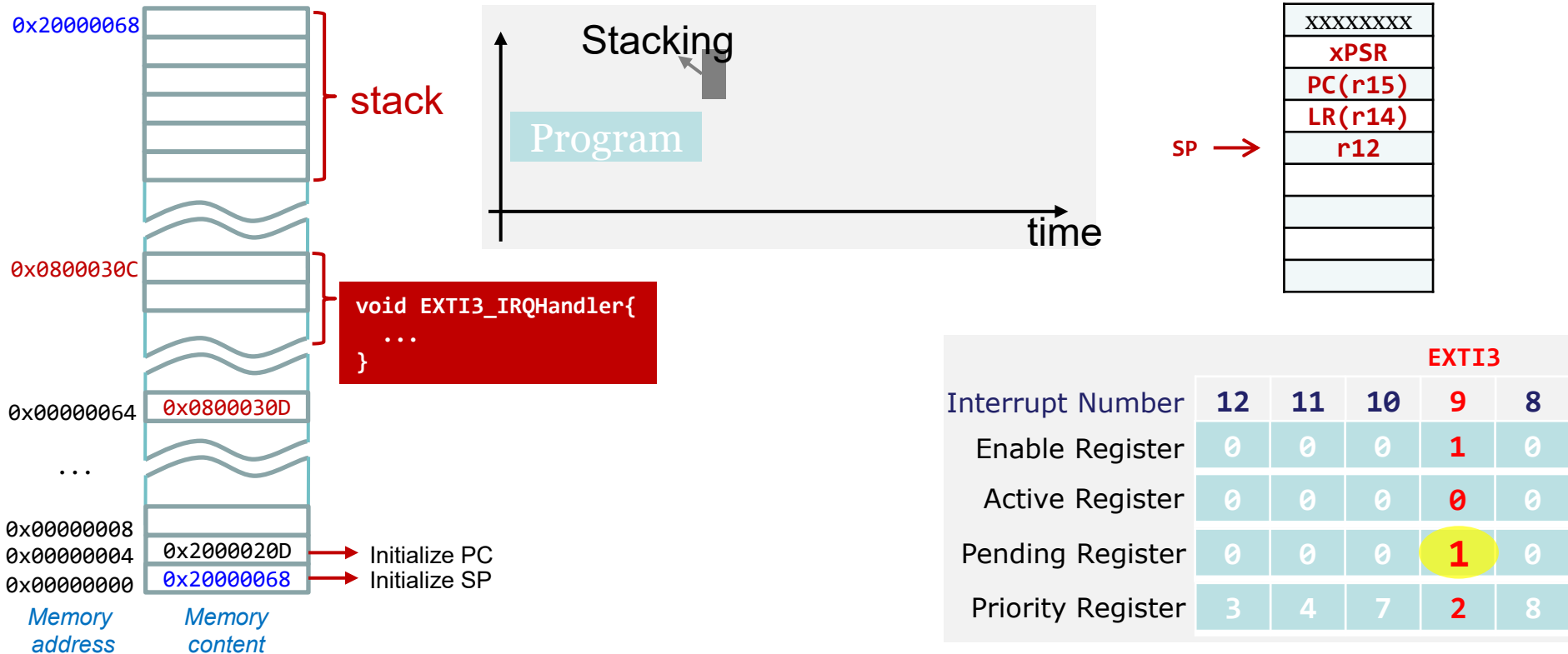
Single Interrupt



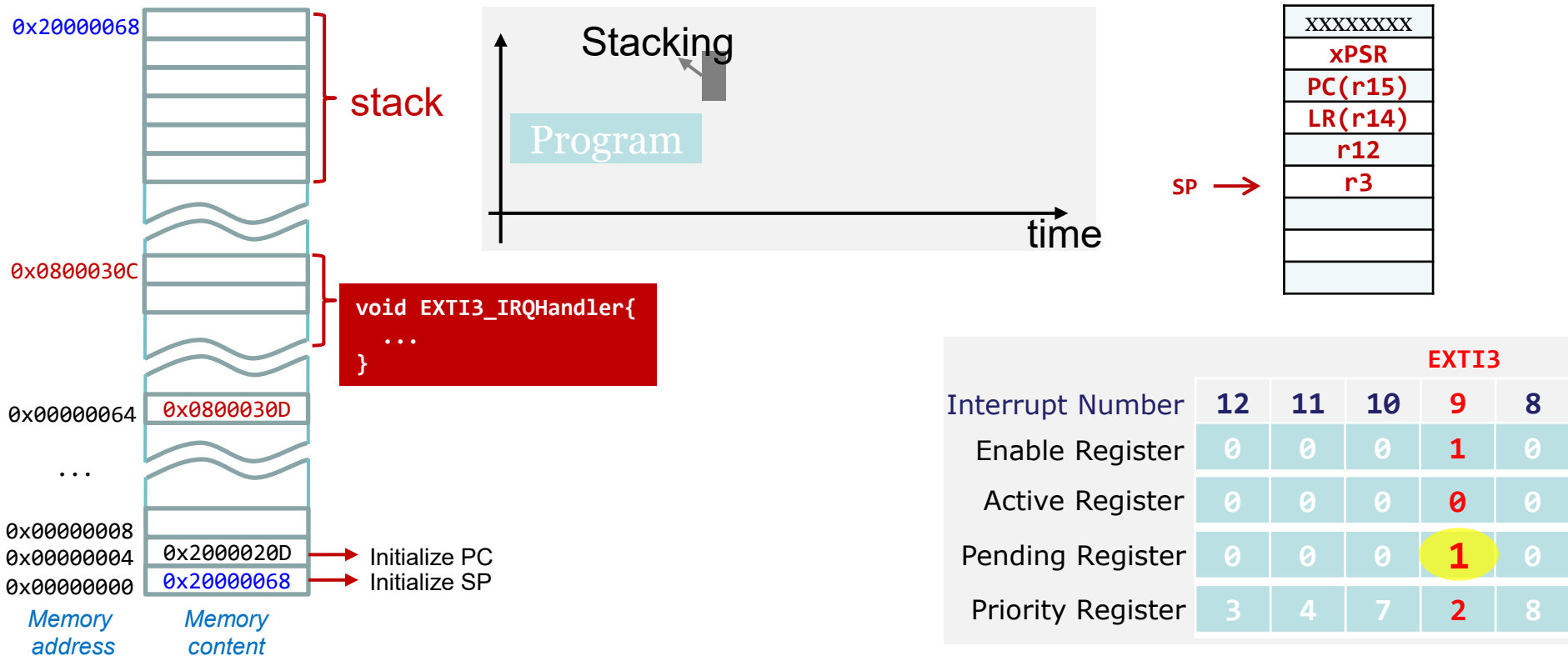
Single Interrupt



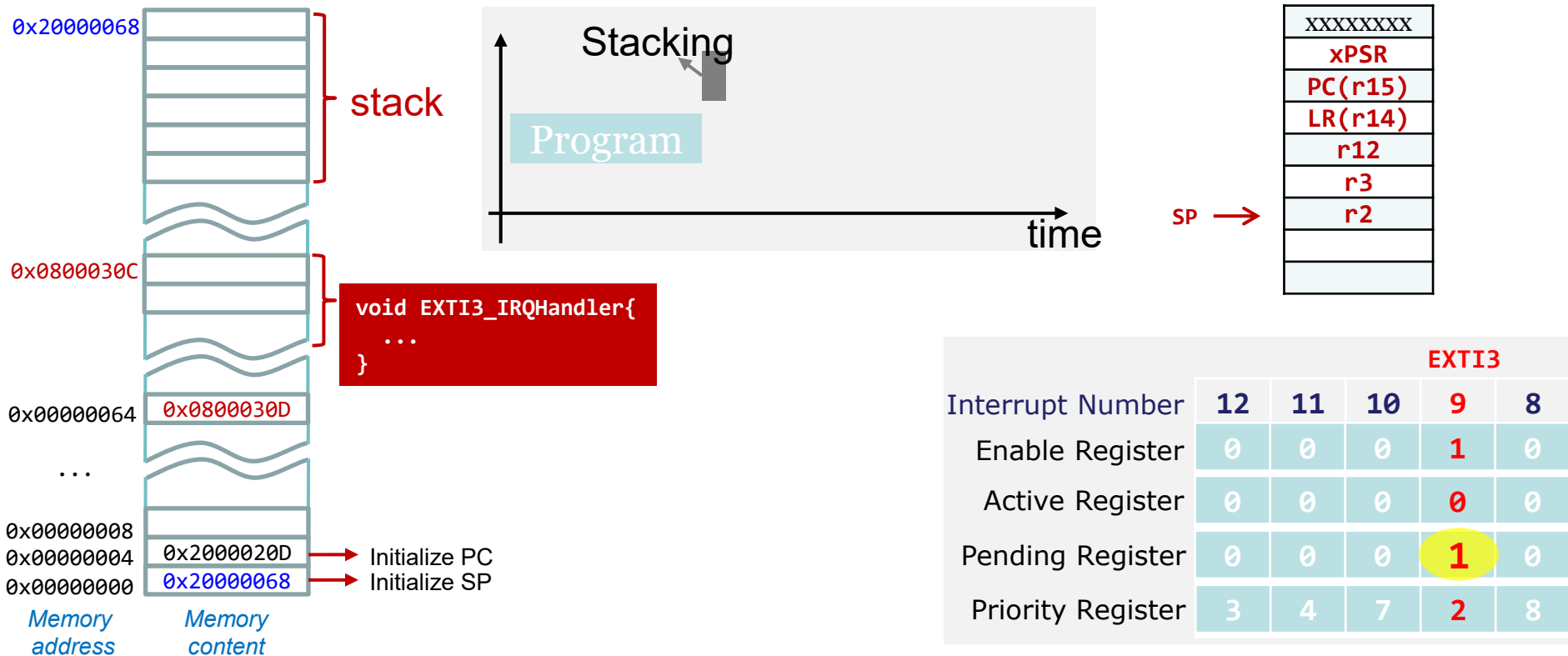
Single Interrupt



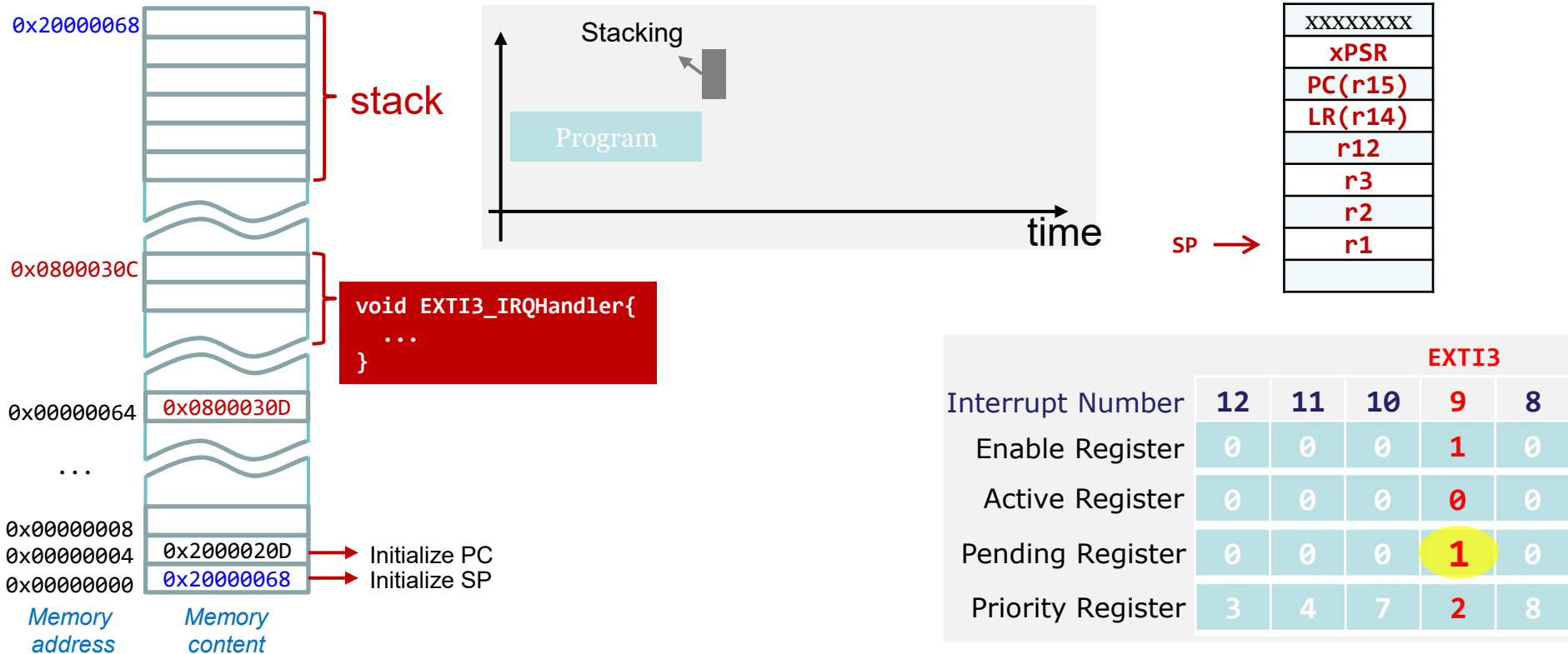
Single Interrupt



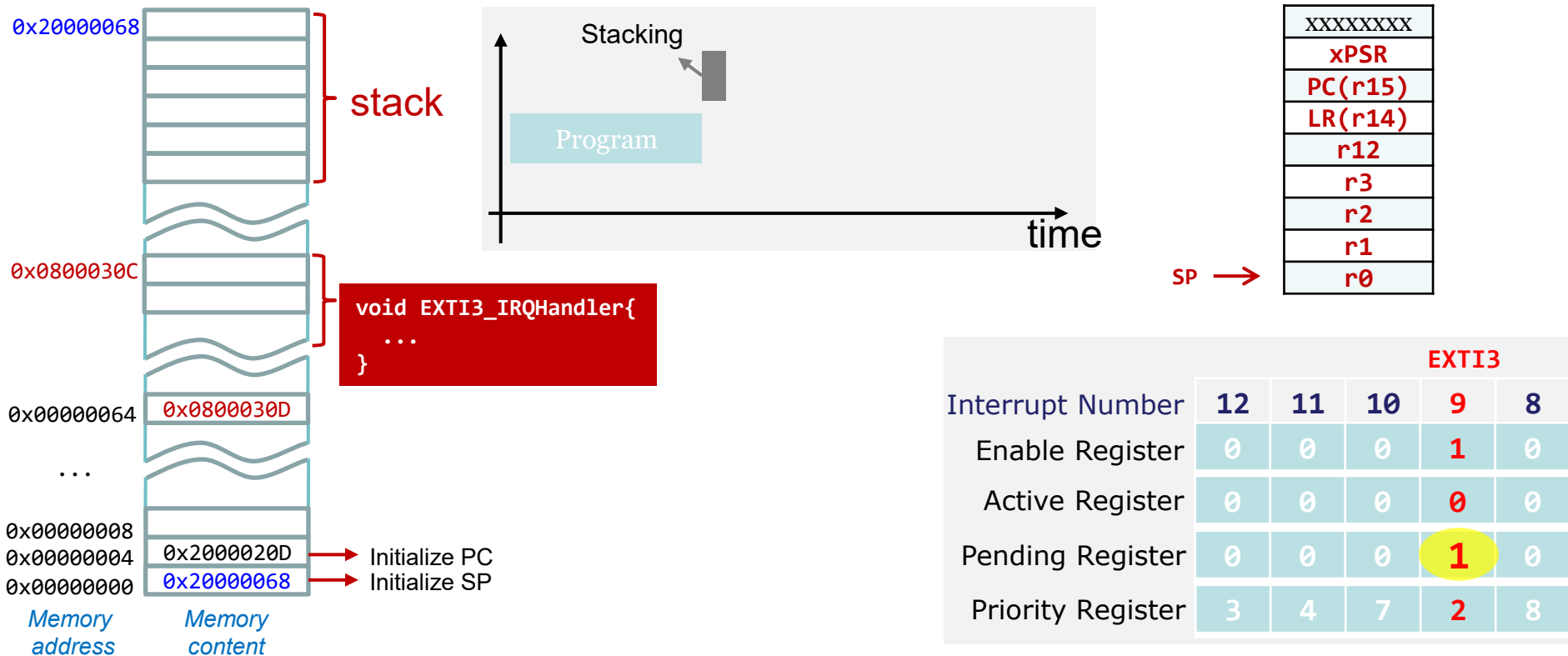
Single Interrupt



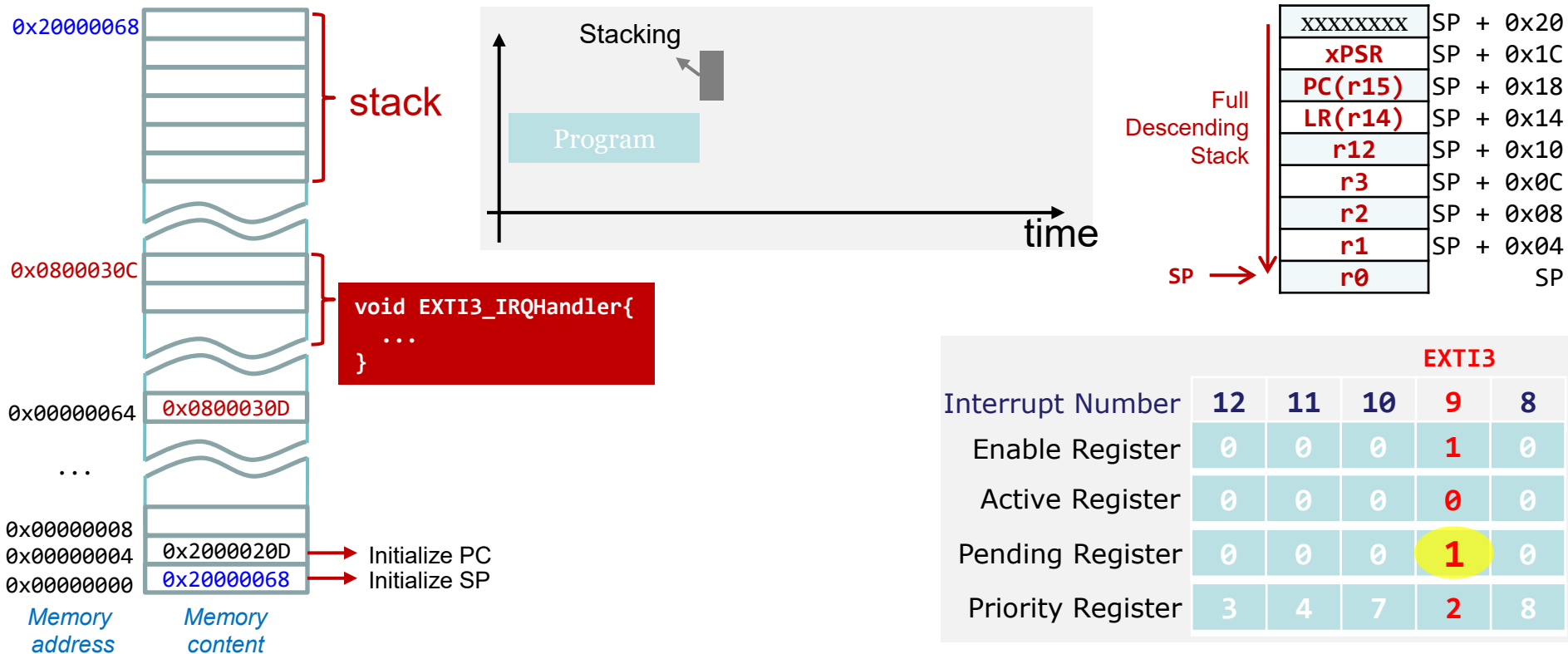
Single Interrupt



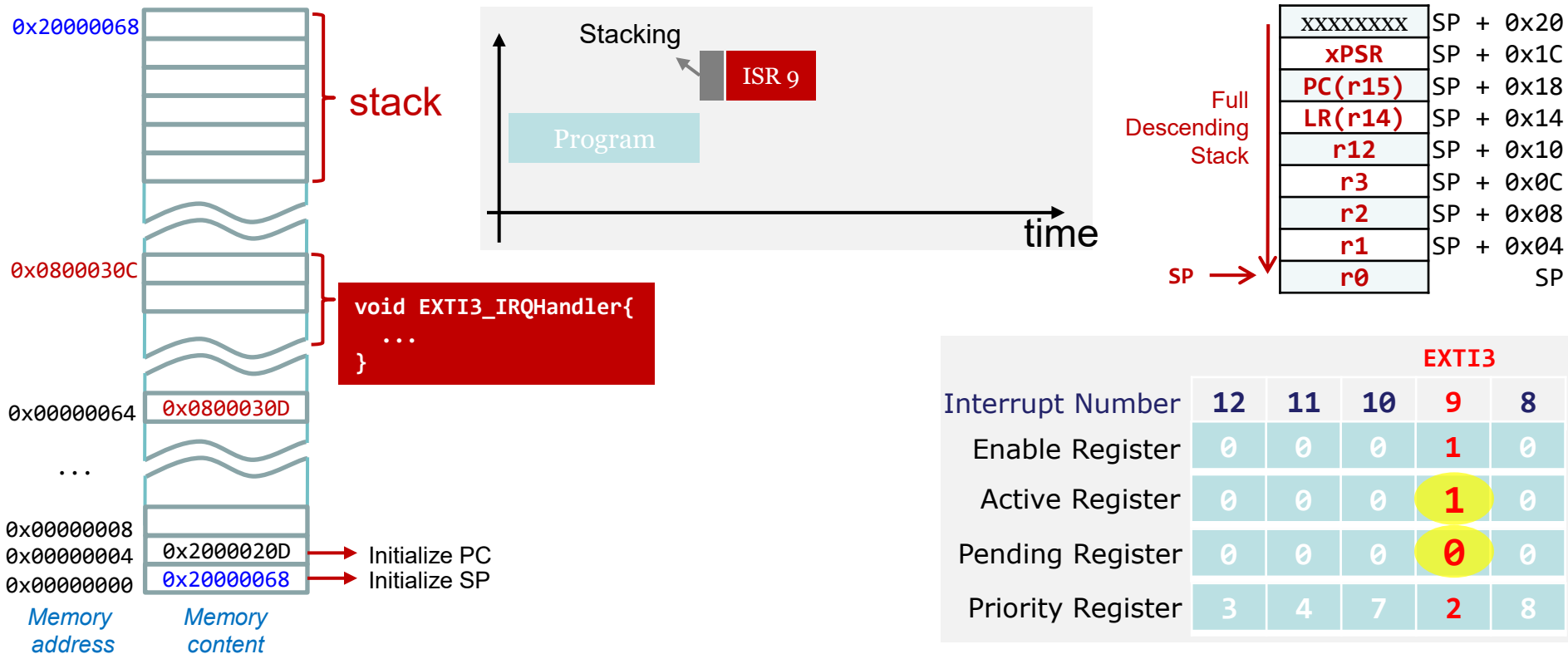
Single Interrupt



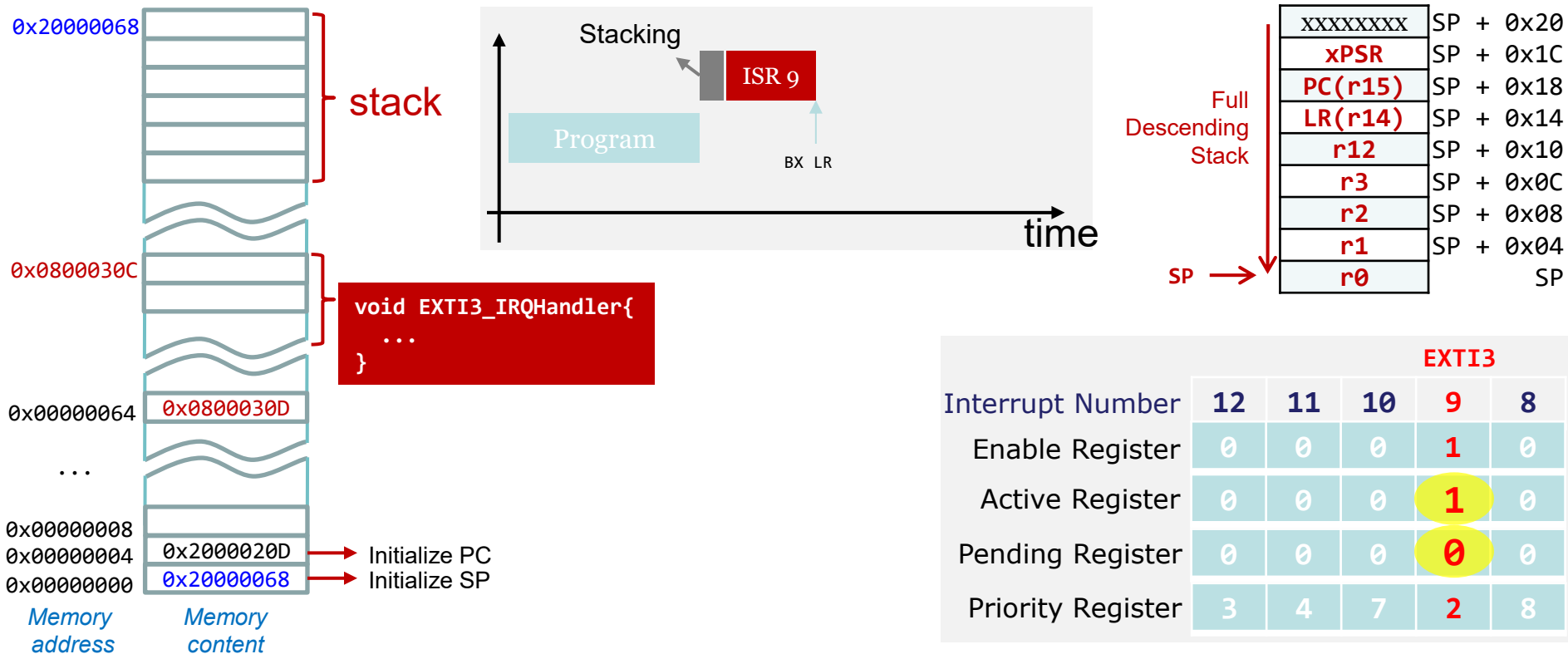
Single Interrupt



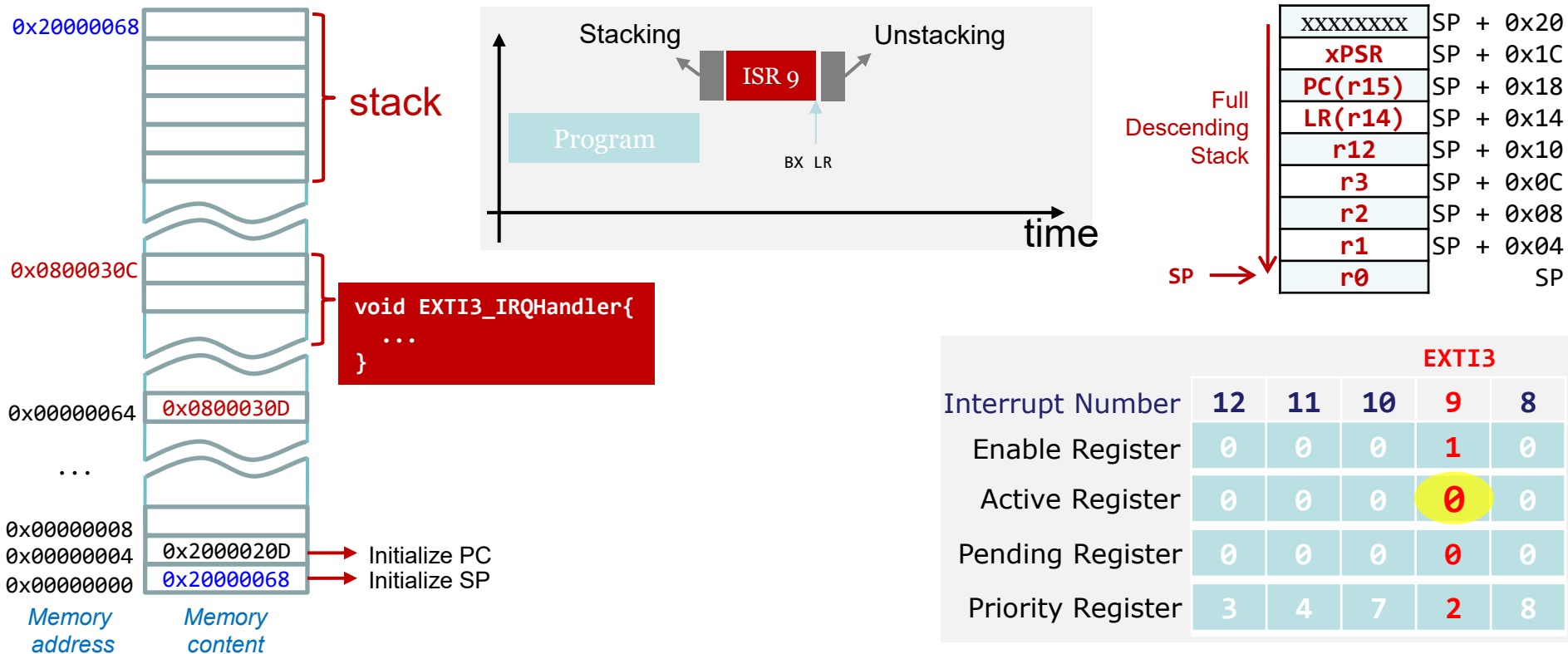
Single Interrupt



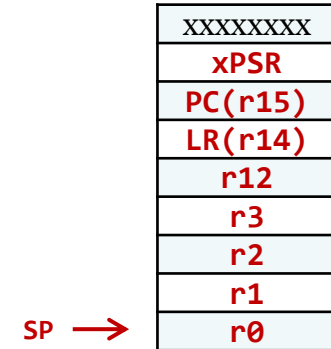
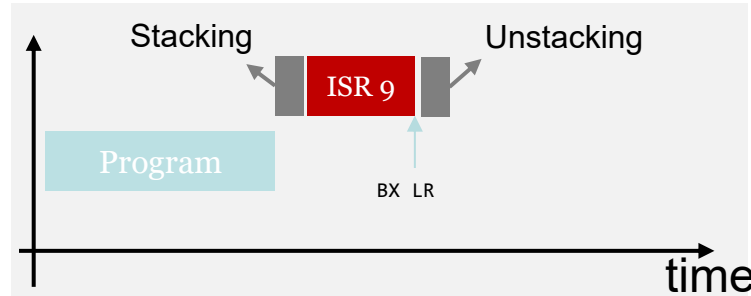
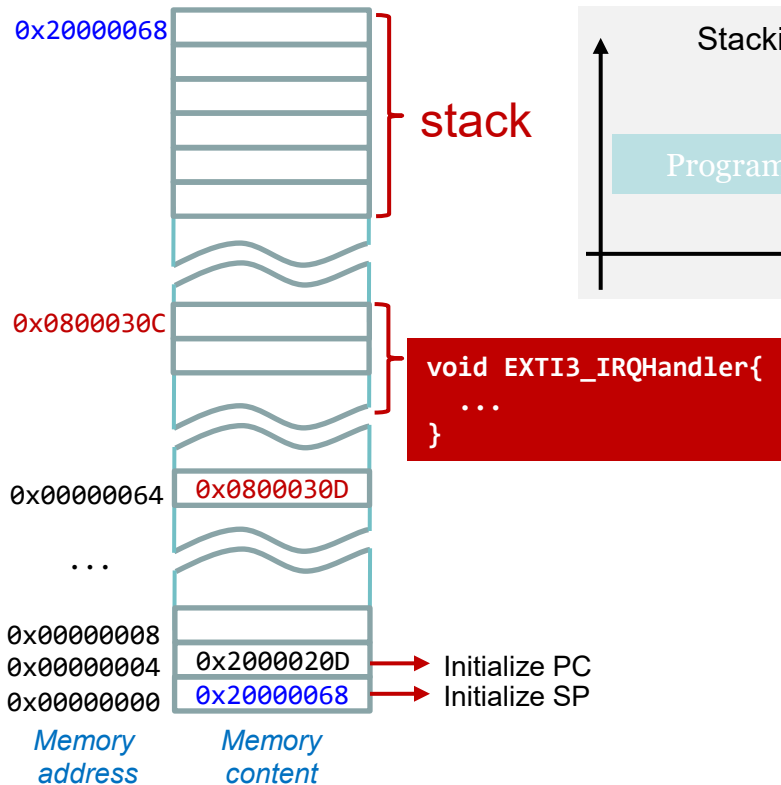
Single Interrupt



Single Interrupt

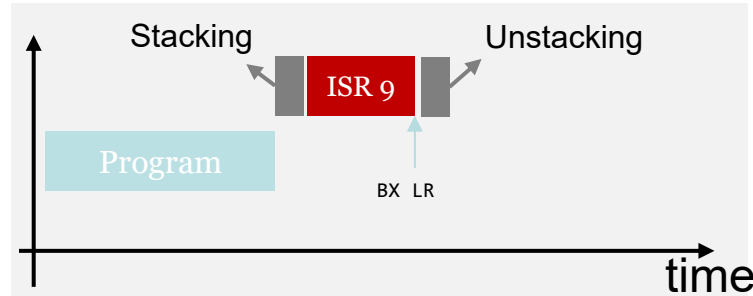
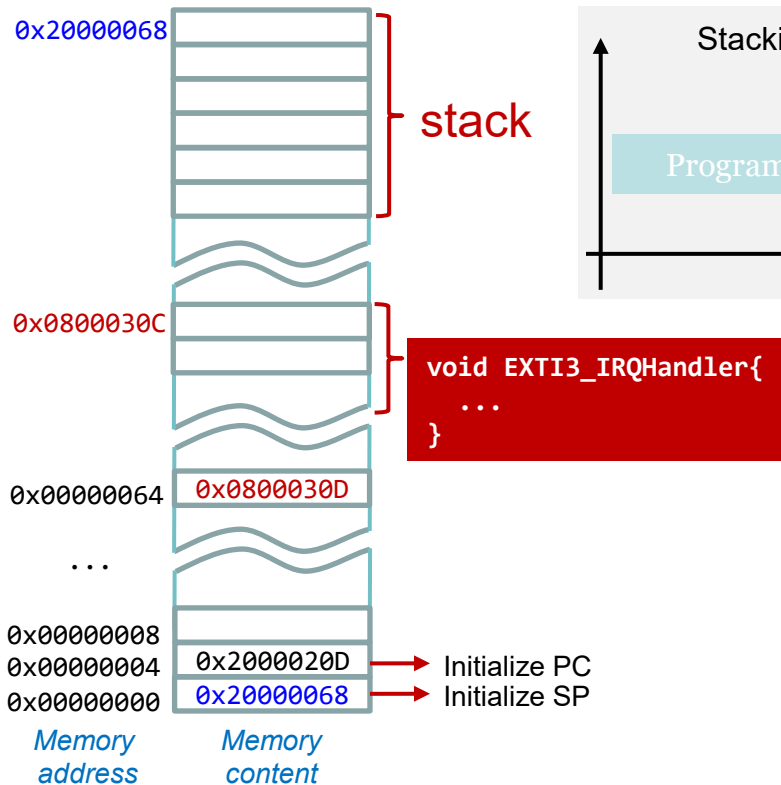


Single Interrupt



	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

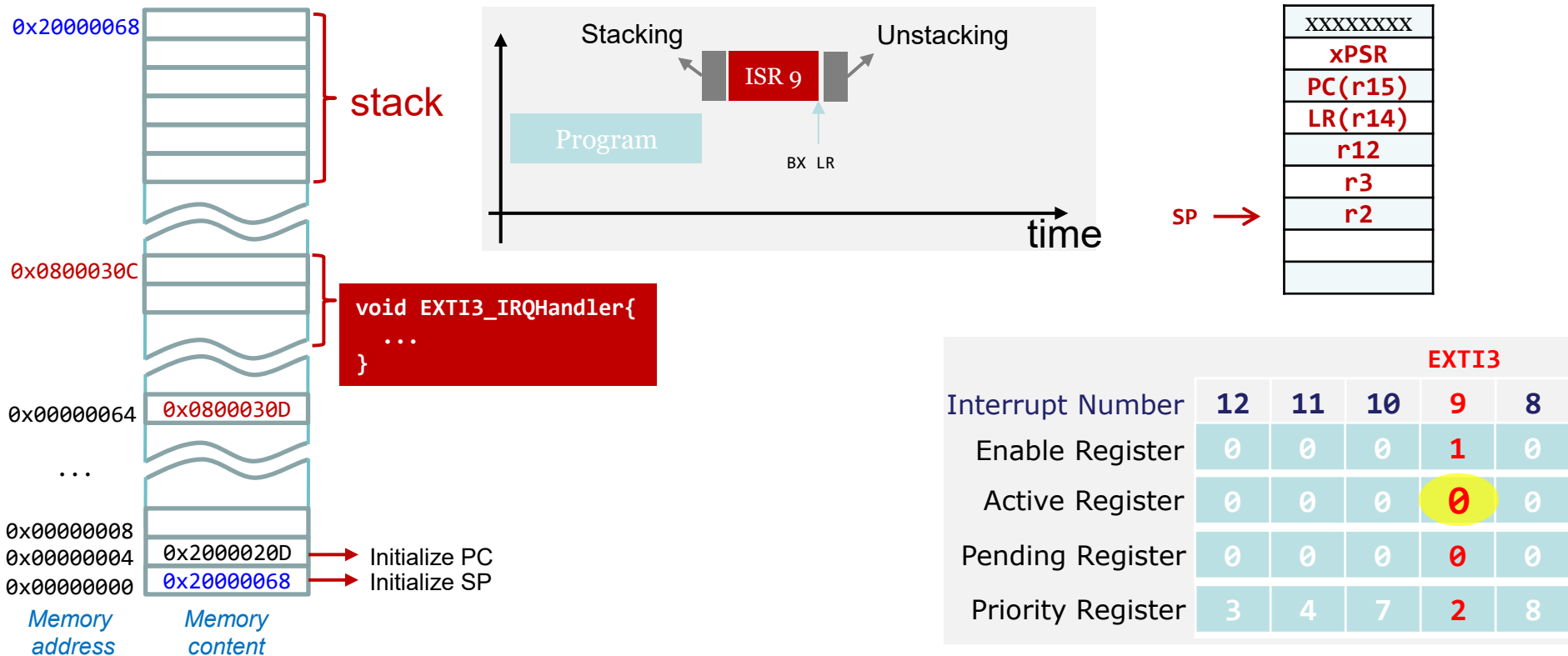


SP →

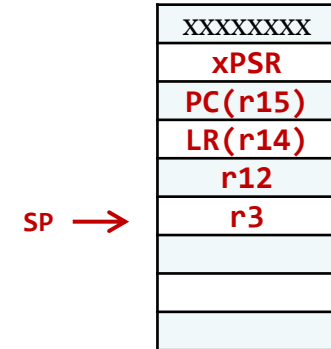
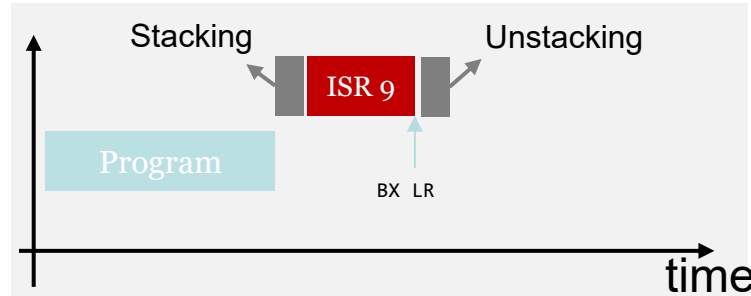
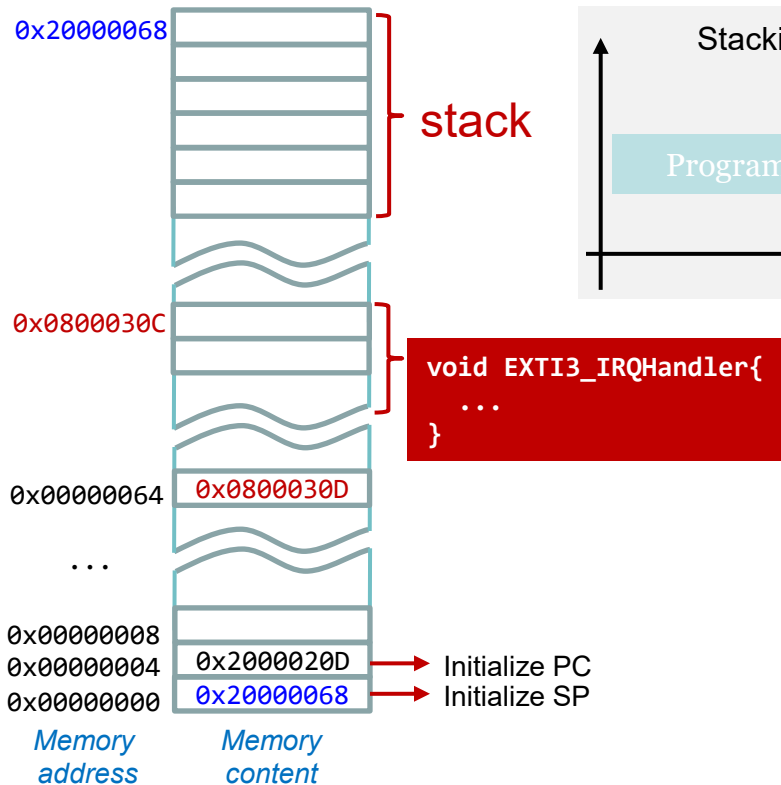
XXXXXXXX
xPSR
PC(r15)
LR(r14)
r12
r3
r2
r1

	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

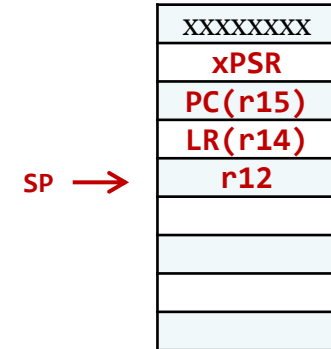
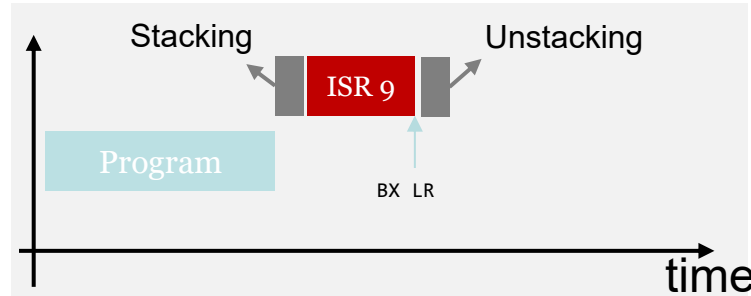
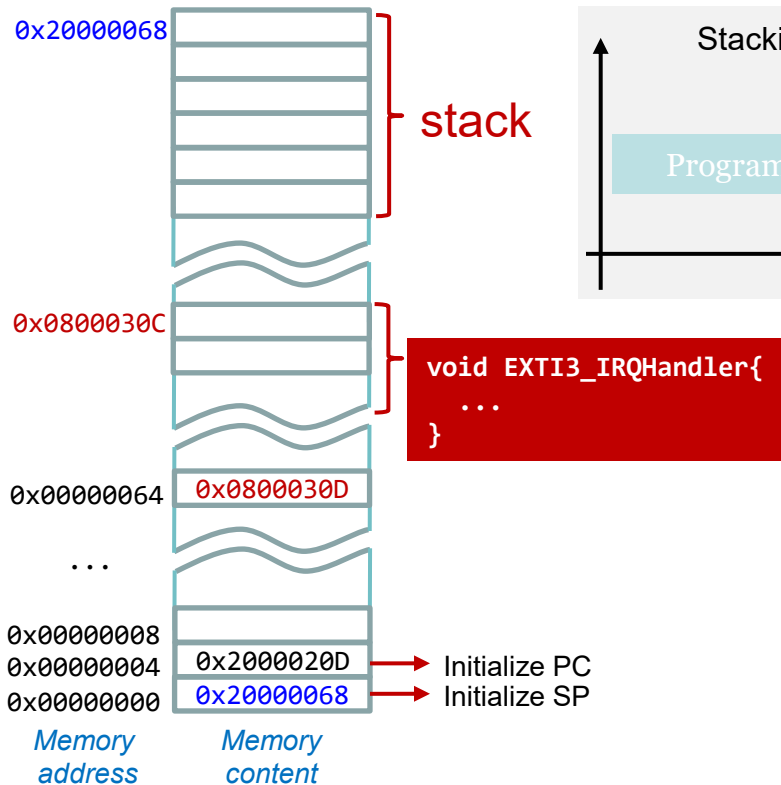


Single Interrupt



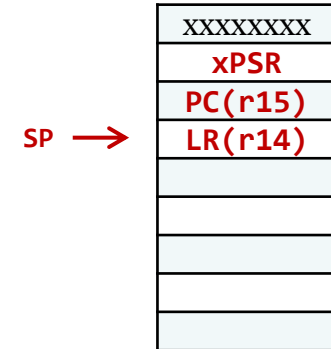
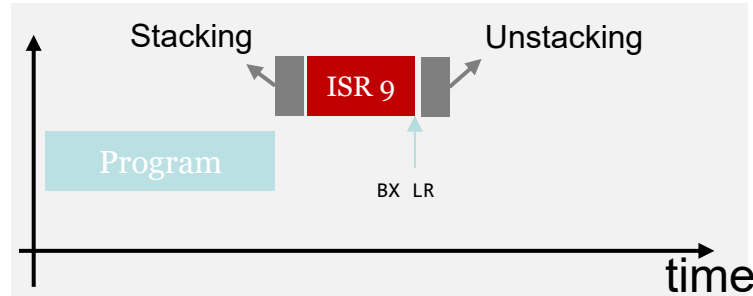
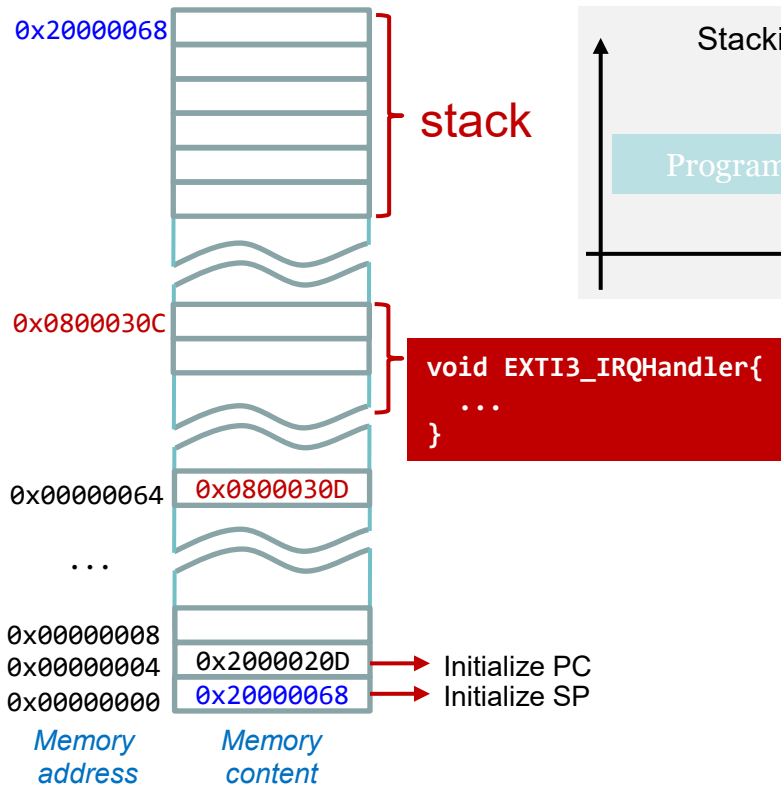
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt



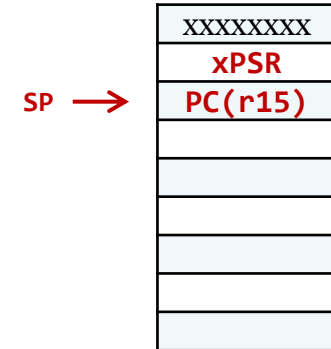
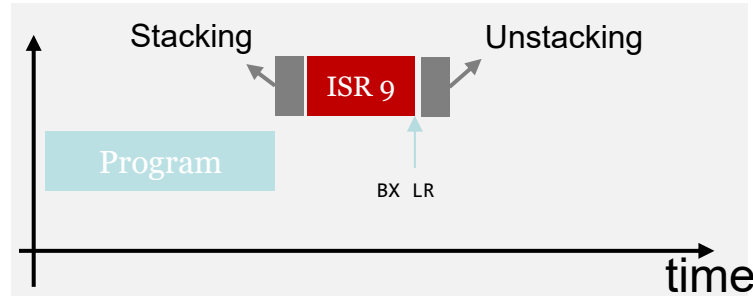
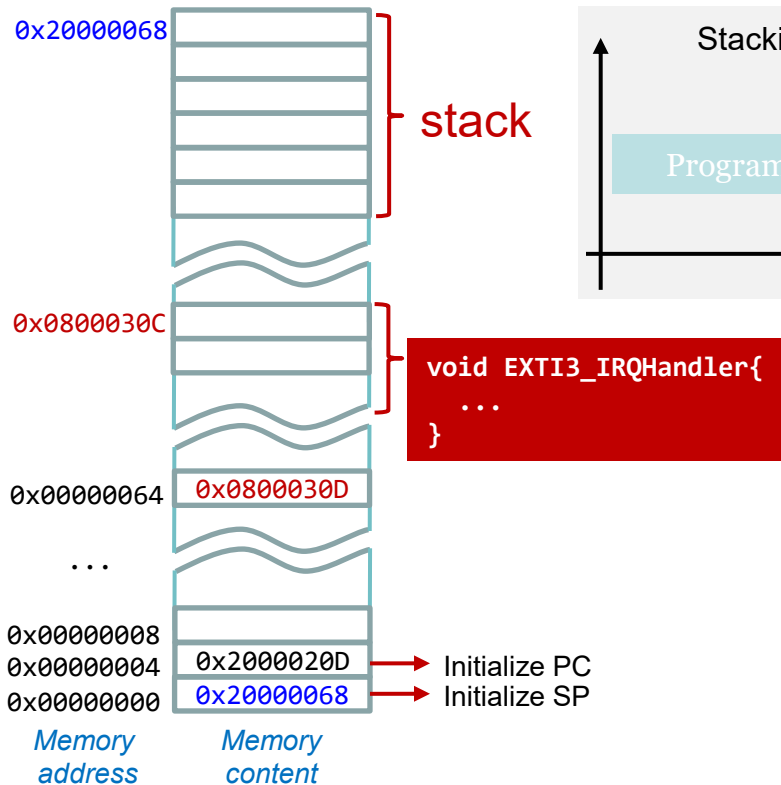
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt



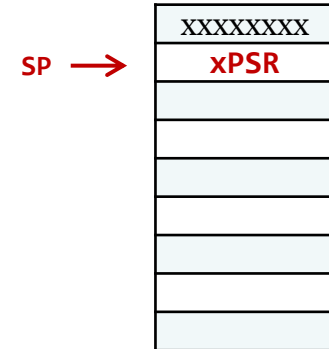
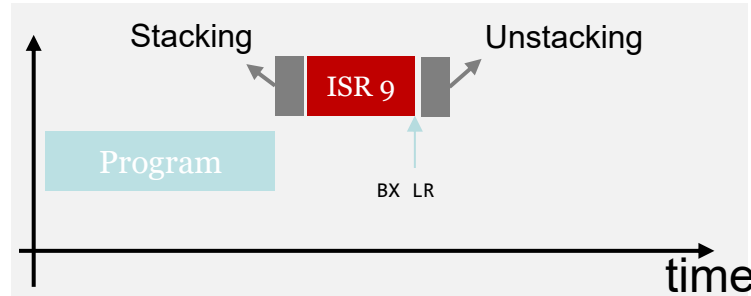
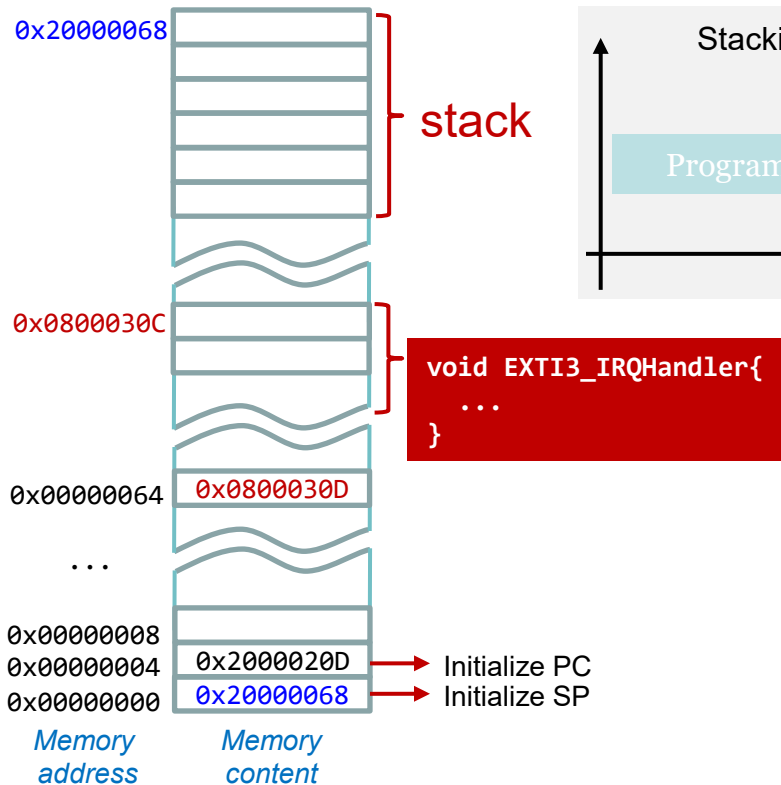
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt



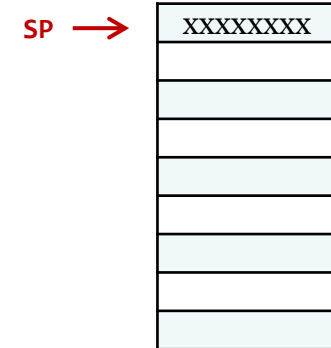
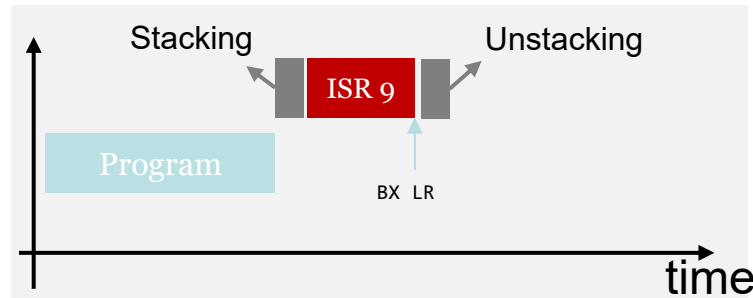
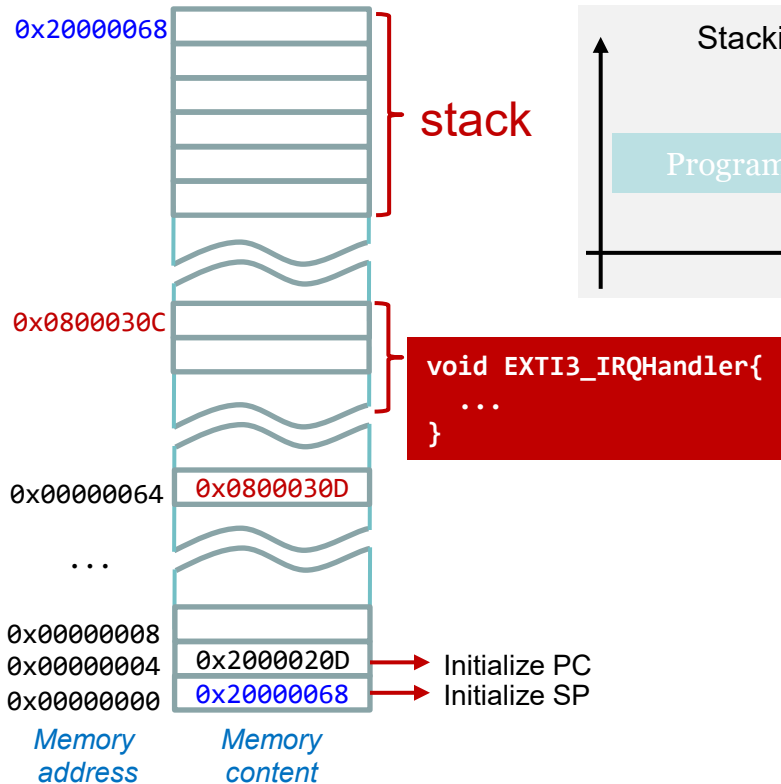
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt



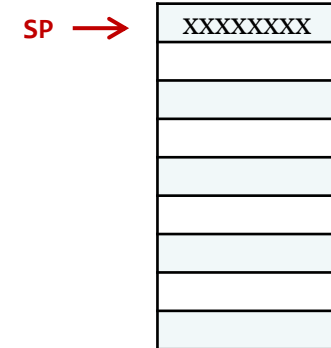
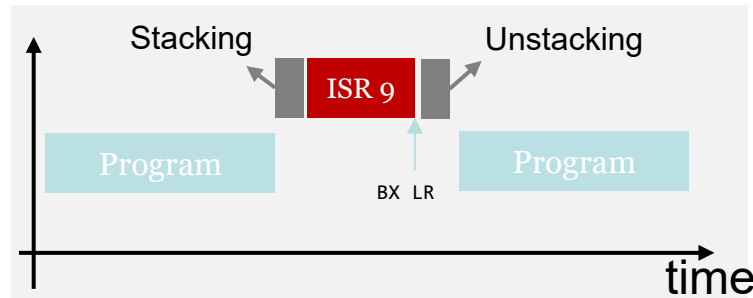
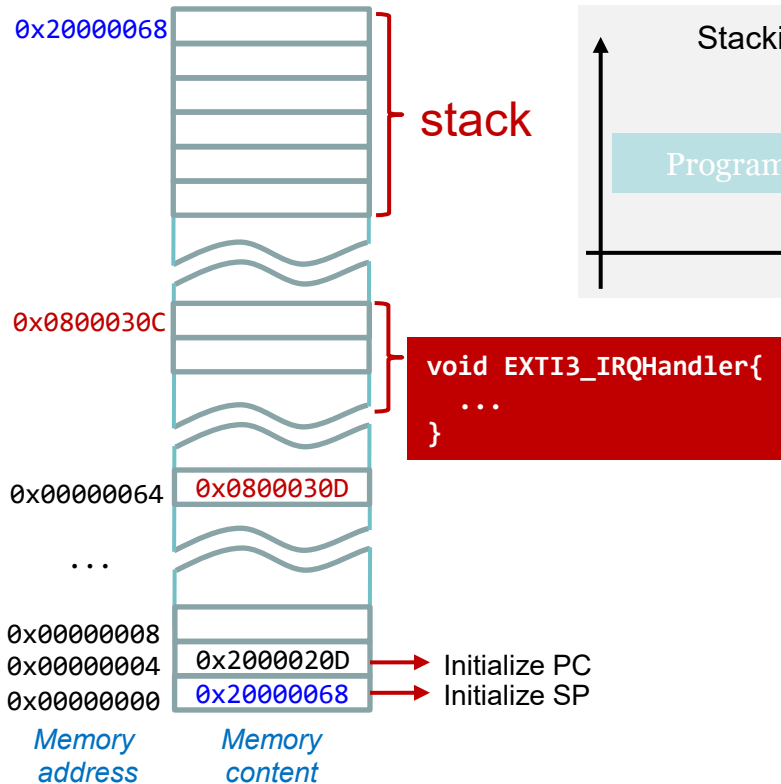
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt



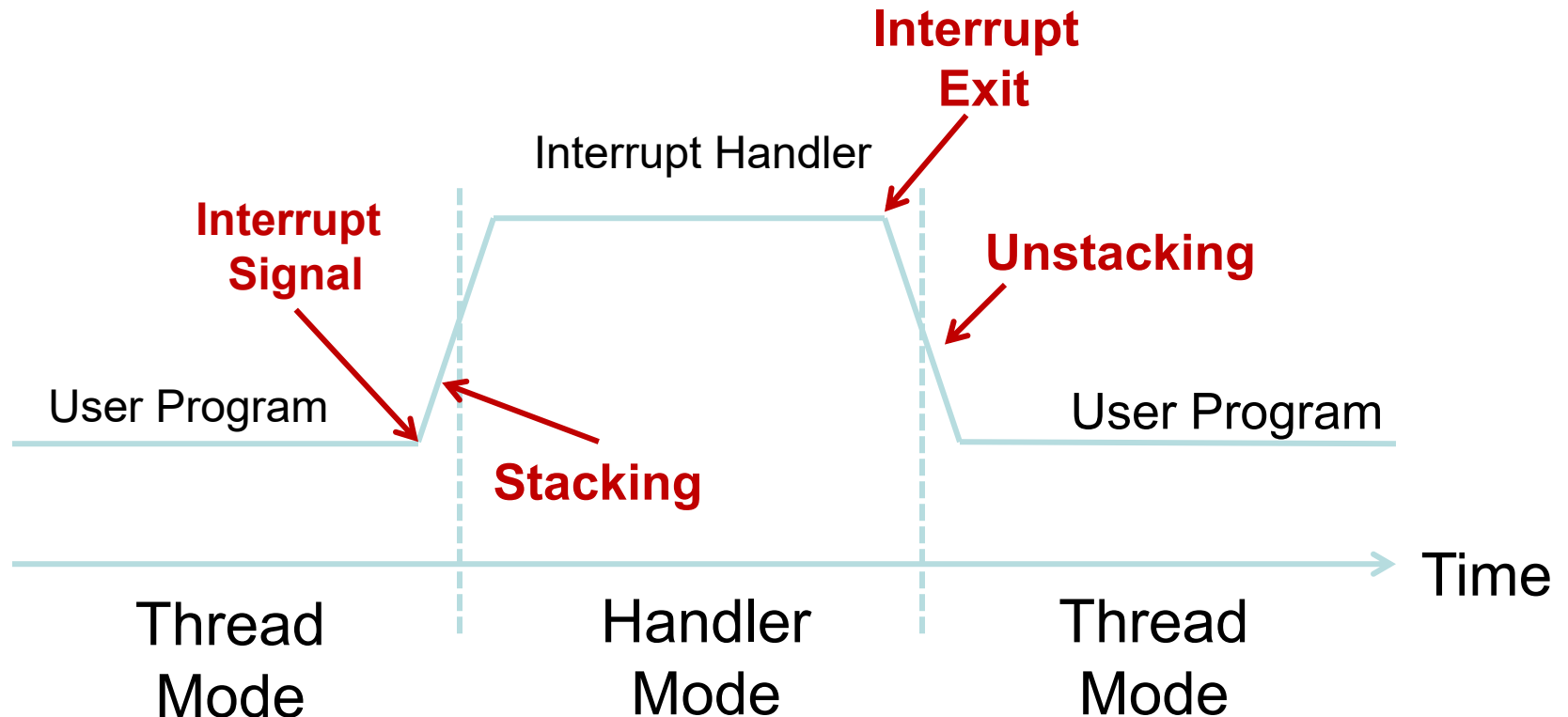
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt



	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

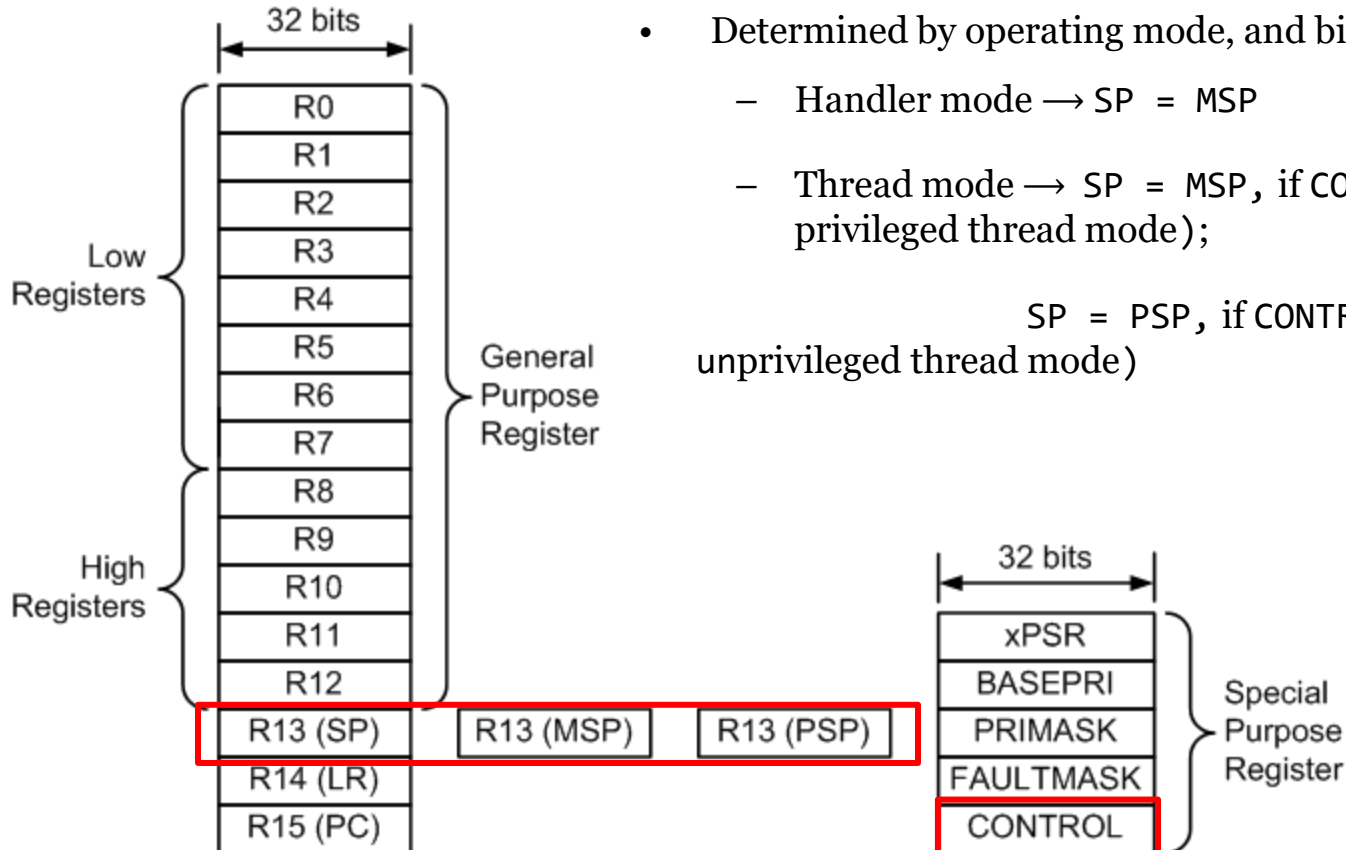
Stacking & Unstacking



Registers

- Two stack pointers: Main SP (MSP) and Process SP (PSP)
- Determined by operating mode, and bit 0 of the CONTROL register
 - Handler mode $\rightarrow$ SP = MSP
 - Thread mode $\rightarrow$ SP = MSP, if CONTROL[0] = 0 (i.e., privileged thread mode);

SP = PSP, if CONTROL[0] = 1 (i.e., unprivileged thread mode)



MSP: Main Stack Pointer

PSP: Process Stack Pointer

Register values in a interrupt service routine

LR = 0xFFFFFFFF9

SP = MSP

ISR always
in handler
mode.

The screenshot shows a debugger interface with the following components:

- Registers Window:**

Register	Value
R0	0x080001A3
R1	0x200005F8
R2	0x00000000
R3	0x20000600
R4	0x080001A3
R5	0x200005F8
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x200005C8
R14 (LR)	0xFFFFFFFF9
R15 (PC)	0x0800017C
xPSR	0x2100000B

Banked registers:

Register	Value
MSP	0x200005C8
PSP	0x00000000

System registers:

Register	Value
BASEPRI	0x00
PRIMASK	0
FAULTMASK	0
CONTROL	0x00

Internal registers:

Register	Value
Mode	Handler
Privilege	Privileged
Stack	MSP
States	2740
Sec	0.00034250
- Disassembly Window:**

```

36: SVC_Handler PROC
37:         EXPORT SVC_Handler
0x0800017A BD30      POP     {r4-r5,pc}
38:         CPSID     I
39:         PUSH      {r4-r8,lr}
0x0800017E E92D41F0  PUSH    {r4-r8,lr}
40:         LDR        r7,[sp,#0x14]
0x08000182 9F05      LDR      r7,[sp,#0x14]
41:         TST        r7,#0x04
0x08000184 F0170F04  TST      r7,#0x04
42:         ITE        EQ
0x08000188 BF0C      ITE        EQ

```
- Logic Analyzer Window:**

```

stm32l1xx_constants.s
startup_stm32l1xx_md.s

33:         POP     {r4-r5,pc}
34:         ENDP
35:
36: SVC_Handler PROC
37:         EXPORT SVC_Handler
38:         CPSID     I
39:         PUSH      {r4-r8,lr}
40:         LDR        r7,[sp,#0x14]
41:         TST        r7,#0x04
42:         ITE        EQ
43:         MRSEQ     r4,msp
44:         MRSNE     r4,psp
45:         LDR        r4,=0x080001A3
46:         LDR        r6,[r4,#0x04]
47:         MOV        r0,r6
48:         BLX        r5
49:         POP        {r4-r8,lr}
50:         CPSIE     I
51:         BX         lr
52:         ENDP

```

Interrupt: Stacking & Unstacking

```

__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
    
```

addr = 0x08000044

addr = 0x0800001C

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08001000
R15(PC)	0x08000044
xPSR	0x21000000
MSP	0x20000200
PSP	0x00000000

XXXXXXXX	0x20000200
	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory

Suppose SysTick interrupt occurs when PC = 0x08000044

__main PROC

...
addr = 0x08000044

...

MOV r3, #0

...

ENDP **addr = 0x0800001C**

SysTick_Handler PROC

EXPORT SysTick_Handler

ADD r3, #1

ADD r4, #1

BX lr

ENDP

R1	1		0x200001FC
R2	2		0x200001F8
R3	3		0x200001F4
R4	4		0x200001F0
R12	12		0x200001EC
R13(SP)	MSP		0x200001E8
R14(LR)	0x08001000		0x200001E4
R15(PC)	0x08000044		0x200001E0
xPSR	0x21000000		0x200001DC
MSP	0x20000200		0x200001D8
PSP	0x00000000		0x200001D4
			0x200001D0
			0x200001CF

Memory

STACKING

```

__main PROC
...
MOV r3, #0
...
ENDP
SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP

```

addr = 0x08000044

addr = 0x0800001C

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFFF9
R15(PC)	0x0800001C
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

xxxxxxx	0x20000200
xPSR 0x21000000	0x200001FC
PC 0x08000044	0x200001F8
LR 0x08001000	0x200001F4
R12 12	0x200001F0
R3 3	0x200001EC
R2 2	0x200001E8
R1 1	0x200001E4
R0 0	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory

STACKING

`__main PROC`

`addr = 0x08000044`

`...`

`MOV r3, #0`

`...`

`ENDP` `addr = 0x0800001C`

`SysTick_Handler PROC`

`EXPORT SysTick_Handler`

`ADD r3, #1`

`ADD r4, #1`

`BX lr`

`ENDP`

**LR = 0xFFFFFFFF9 to
indicate MSP is used.**

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFFF9
R15(PC)	0x0800001C

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Interrupt: Stacking & Unstacking

```

__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
    
```

addr = 0x08000044

addr = 0x0800001C

R0	0	XXXXXXXXXX	0x20000200
R1	1	xPSR 0x21000000	0x200001FC
R2	2	PC 0x08000044	0x200001F8
R3	4	LR 0x08001000	0x200001F4
R4	4	R12 12	0x200001F0
R12	12	R3 3	0x200001EC
R13(SP)	MSP	R2 2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1 1	0x200001E4
R15(PC)	0x0800001C	R0 0	0x200001E0
			0x200001DC
xPSR	0x21000000		0x200001D8
MSP	0x200001E0		0x200001D4
PSP	0x00000000		0x200001D0
			0x200001CF

Memory

Interrupt: Stacking & Unstacking

```

__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP

```

addr = 0x08000044

addr = 0x0800001C

R0	0	XXXXXXXXXX	0x20000200
R1	1	xPSR 0x21000000	0x200001FC
R2	2	PC 0x08000044	0x200001F8
R3	4	LR 0x08001000	0x200001F4
R4	5	R12 12	0x200001F0
R12	12	R3 3	0x200001EC
R13(SP)	MSP	R2 2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1 1	0x200001E4
R15(PC)	0x08000020	R0 0	0x200001E0
			0x200001DC
xPSR	0x21000000		0x200001D8
MSP	0x200001E0		0x200001D4
PSP	0x00000000		0x200001D0
			0x200001CF

Memory

Interrupt: Stacking & Unstacking

```

__main PROC
...
MOV r3, #0
...
ENDP

```

addr = 0x08000044

addr = 0x0800001C

```

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP

```

LR = 0xFFFFFFFF9 to
indicate MSP is used.

R0	0	XXXXXXX	0x20000200
R1	1	xPSR	0x21000000
R2	2	PC	0x08000044
R3	4	LR	0x08001000
R4	5	R12	12
R12	12	R3	3
R13(SP)	MSP	R2	2
R14(LR)	0xFFFFFFFF9	R1	1
R15(PC)	0x08000024	R0	0
xPSR	0x21000000		
MSP	0x200001E0		
PSP	0x00000000		

Memory

UNSTACKING

```

__main PROC
...
MOV r3, #0
...
ENDP

```

addr = 0x08000044

addr = 0x0800001C

```

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP

```

LR = 0xFFFFFFFF9 to
indicate MSP is used.

R0	0		XXXXXXXXXX	0x20000200
R1	1	xPSR	0x21000000	0x200001FC
R2	2	PC	0x08000044	0x200001F8
R3	4	LR	0x08001000	0x200001F4
R4	5	R12	12	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x08000024	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory

`__main PROC`

`addr = 0x08000044`

`...`

`MOV r3, #0`

`...`

`ENDP` `addr = 0x0800001C`

`SysTick_Handler PROC`

`EXPORT SysTick_Handler`

`ADD r3, #1`

`ADD r4, #1`

`BX lr`

`ENDP`

**Note the new value
of R3 is lost!!!**

R0	0
R1	1
R2	2
R3	3
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0x08001000
R15(PC)	0x08000044
xPSR	0x21000000
MSP	0x20000200
PSP	0x00000000

XXXXXXXX	0x20000200
	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
...
MOV r3, #0
...
ENDP
```

addr = 0x08000044

addr = 0x0800001C

```
SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r3, #1
ADD r4, #1
BX lr
ENDP
```

**The Main program
resumes!!!**

R0	0
R1	1
R2	2
R3	3
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0x08001000
R15(PC)	0x08000044
xPSR	0x21000000
MSP	0x20000200
PSP	0x00000000

XXXXXXXX	0x20000200
	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory



What if ISR calls subroutine?

Interrupt: Stacking & Unstacking

```

__main PROC
...
MOV r3, #0
...
ENDP

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r4, #1
BL sine
BX lr
ENDP
    
```

addr = 0x08000044

addr = 0x0800001C

R0	0	XXXXXXXXXX	0x20000200
R1	1		0x200001FC
R2	2		0x200001F8
R3	3		0x200001F4
R4	4		0x200001F0
R12	12		0x200001EC
R13(SP)	MSP		0x200001E8
R14(LR)	0x08001000		0x200001E4
R15(PC)	0x08000044		0x200001E0
xPSR	0x21000000		0x200001DC
MSP	0x20000200		0x200001D8
PSP	0x00000000		0x200001D4
			0x200001D0
			0x200001CF

Memory

STACKING

```

__main PROC
...
MOV r3, #0
...
ENDP

```

addr = 0x08000044

addr = 0x0800001C

```

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r4, #1
BL sine
BX lr
ENDP

```

LR = 0xFFFFFFFF9 to
indicate MSP is used.

R0	0		XXXXXXXXXX	0x20000200
R1	1	xPSR	0x21000000	0x200001FC
R2	2	PC	0x00000044	0x200001F8
R3	3	SP	0x20000200	0x200001F4
R4	4	LR	0x08001000	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x0800001C	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory

STACKING

```

__main PROC
...
MOV r3, #0
...
ENDP
SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r4, #1
BL sine
BX lr
ENDP

```

addr = 0x08000044

addr = 0x0800001C

R0	0		XXXXXXXX	0x20000200
R1	1	xPSR	0x21000000	0x200001FC
R2	2	PC	0x00000044	0x200001F8
R3	3	SP	0x20000200	0x200001F4
R4	5	LR	0x08001000	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x0800001C	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory

STACKING

```

__main PROC
...
MOV r3, #0
...
ENDP

```

addr = 0x08000044

addr = 0x0800001C

```

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r4, #1
BL sine
BX lr
ENDP

```

BL sine
Updates LR register

R0	0		XXXXXXXX	0x20000200
R1	1	xPSR	0x21000000	0x200001FC
R2	2	PC	0x00000044	0x200001F8
R3	3	SP	0x20000200	0x200001F4
R4	5	LR	0x08001000	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0xFFFFFFFF9	R1	1	0x200001E4
R15(PC)	0x08000020	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory

STACKING

__main PROC

...

MOV r3, #0

...

ENDP

addr = 0x08000044

addr = 0x0800001C

SysTick_Handler PROC

EXPORT SysTick_Handler

ADD r4, #1

BL sine

BX lr

ENDP

BL sine

Updates LR register

R0 0

R1 1

R2 2

R3 3

R4 5

R12 12

R13(SP) MSP

R14(LR) 0x08000024

R15(PC) 0x08000020

xPSR 0x21000000

MSP 0x200001E0

PSP 0x00000000

xPSR 0x21000000 0x200001FC

PC 0x00000044 0x200001F8

SP 0x20000200 0x200001F4

LR 0x08001000 0x200001F0

R3 3 0x200001EC

R2 2 0x200001E8

R1 1 0x200001E4

R0 0 0x200001E0

0x200001DC

0x200001D8

0x200001D4

0x200001D0

0x200001CF

Memory

UNSTACKING won't occurs!

```

__main PROC
...
MOV r3, #0
...
ENDP

```

addr = 0x08000044

addr = 0x0800001C

```

SysTick_Handler PROC
EXPORT SysTick_Handler
ADD r4, #1
BL sine
BX lr
ENDP

```

**BL sine
Updates LR register**

R0	0		XXXXXXXX	0x20000200
R1	1	xPSR	0x21000000	0x200001FC
R2	2	PC	0x00000044	0x200001F8
R3	3	SP	0x20000200	0x200001F4
R4	5	LR	0x08001000	0x200001F0
R12	12	R3	3	0x200001EC
R13(SP)	MSP	R2	2	0x200001E8
R14(LR)	0x08000024	R1	1	0x200001E4
R15(PC)	0x08000024	R0	0	0x200001E0
				0x200001DC
xPSR	0x21000000			0x200001D8
MSP	0x200001E0			0x200001D4
PSP	0x00000000			0x200001D0
				0x200001CF

Memory

Interrupt: Stacking & Unstacking

```

__main PROC
...
    addr = 0x08000044
    MOV r3, #0
...
    ENDP
    addr = 0x0800001C

SysTick_Handler PROC
    EXPORT SysTick_Handler
    PUSH {lr}
    ADD r4, #1
    BL sine
    POP {lr}
    BX lr
    ENDP
    
```

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFFF9
R15(PC)	0x0800001C

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

xPSR	0x21000000	0x200001FC
PC	0x00000044	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Fix the bug!